

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Corso di preparazione per la selezione territoriale delle Olimpiadi di Informatica

Lezione 7. Grafi

Giuseppe F. Italiano

21 aprile 2022 (2775esimo Natale di Roma!)

LUISS



Programma di oggi

1. Correzione compiti a casa
2. Grafi
3. Un problema dalle Territoriali (2015): Rispetta i versi (Rivisitato!)
4. Un problema dalle Territoriali (2004/2005): Sunnydale
5. Un problema dalle Gator (2016): Monete a posto
6. Esercizi Olimpiadi (compiti da fare a casa)

Correzione compiti a casa



Compiti a casa

- Rescaling sequence: https://training.olinfo.it/#/task/ois_rescaling/statement
- Studio amico: https://training.olinfo.it/#/task/abc_studioamico/statement
- Battaglia in pizzeria: https://training.olinfo.it/#/task/mat_pizzeria/statement
- Fibonacci colonies:
https://training.olinfo.it/#/task/ois_fibonaccibug/statement
- Compulsive shopping:
https://training.olinfo.it/#/task/ois_shopping/statement

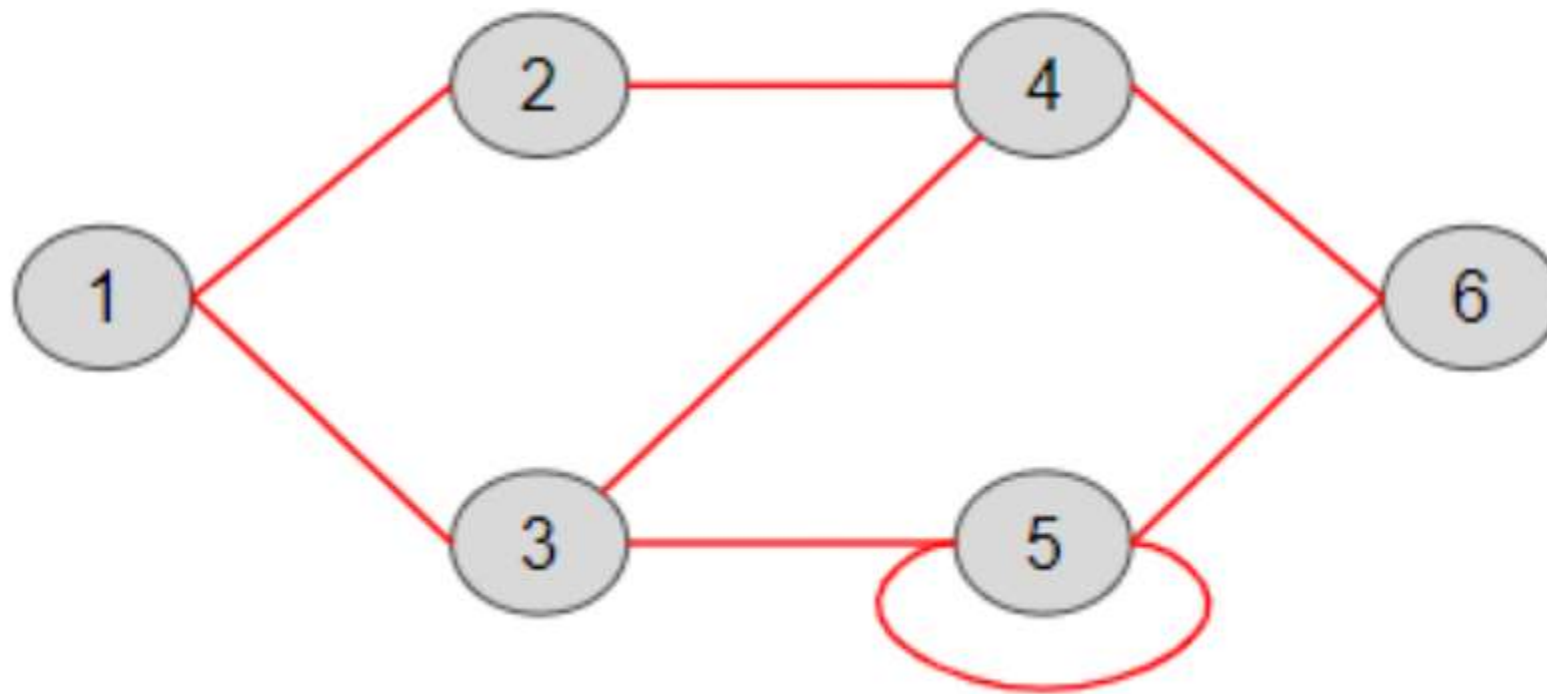
Solo 2 raccomandazioni...



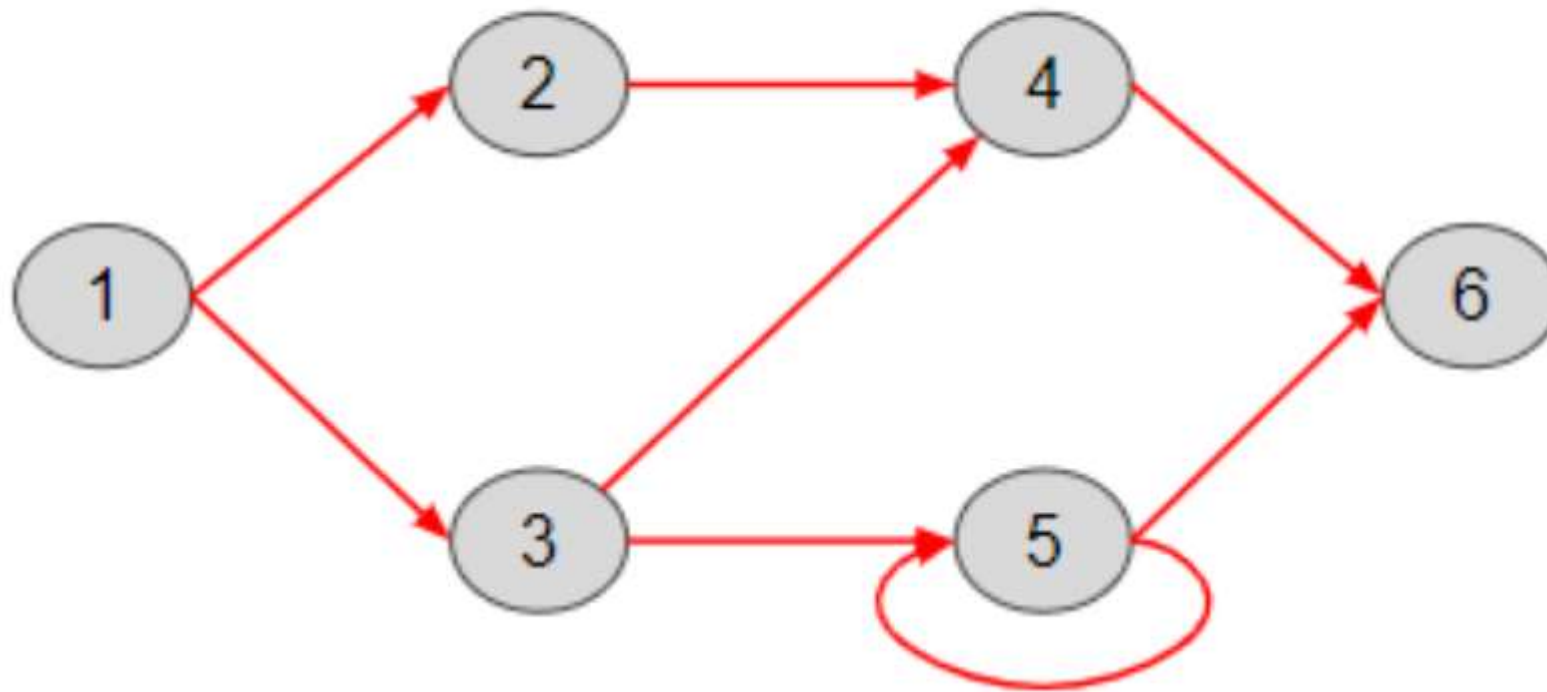
Grafi



GRAFO NON ORIENTATO



GRAFO ORIENTATO

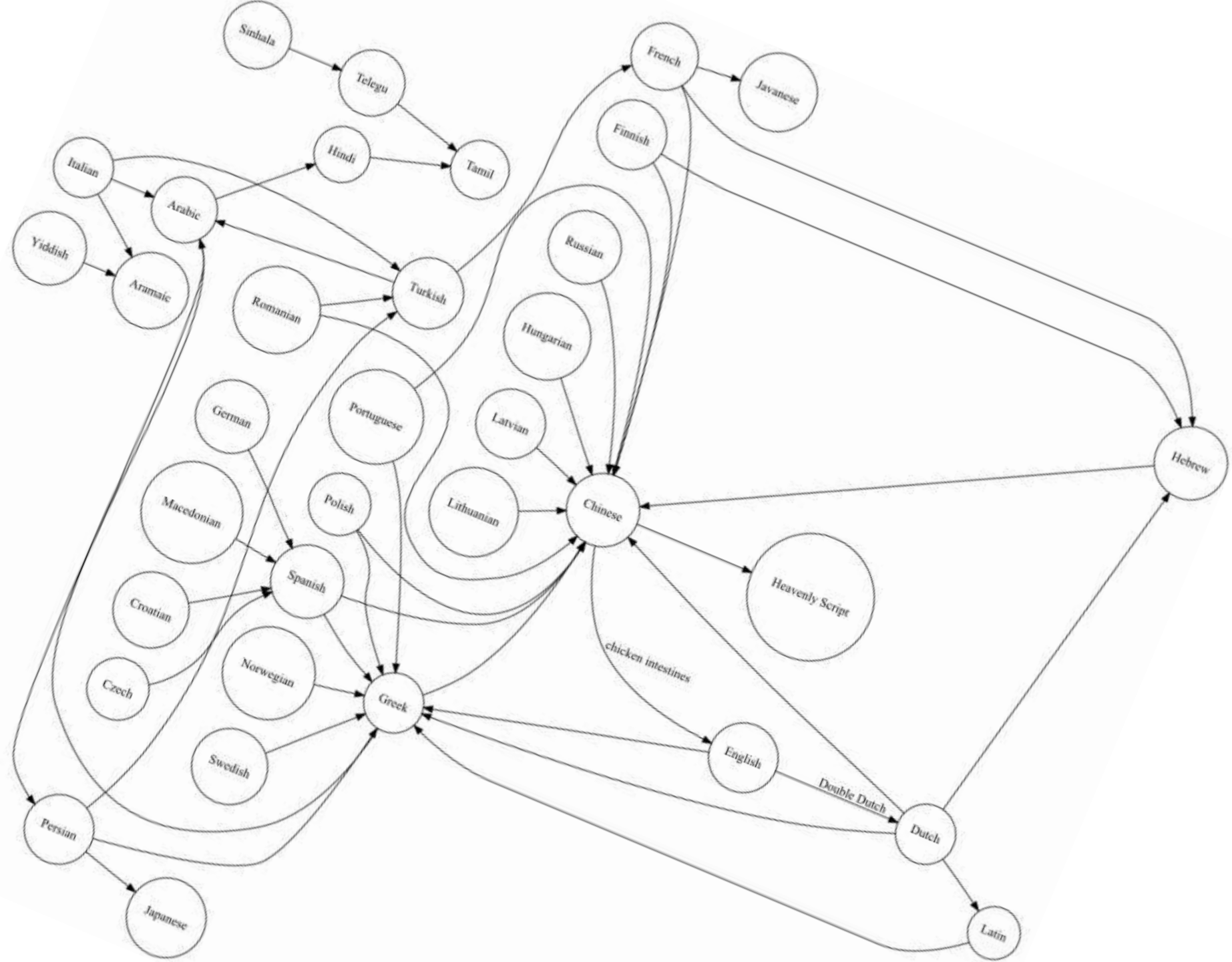


Abilene Federal/Research Network Peers







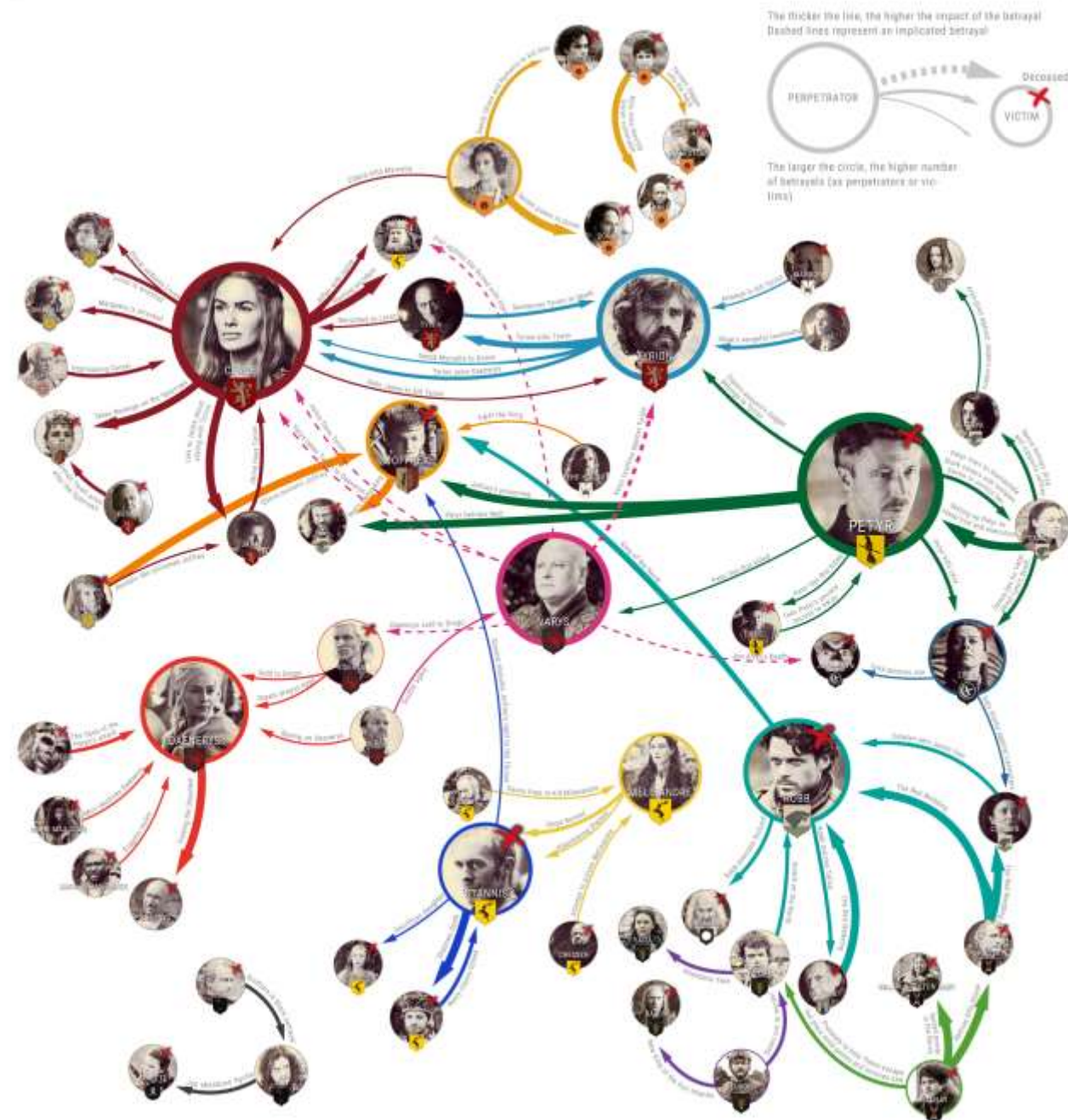


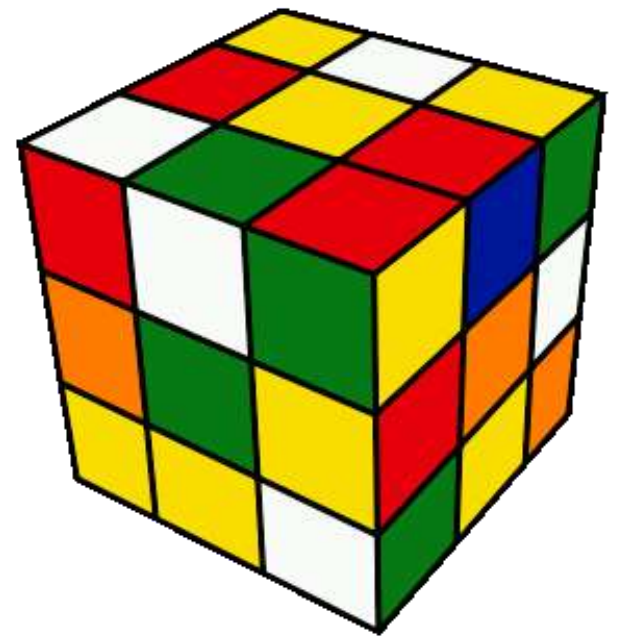
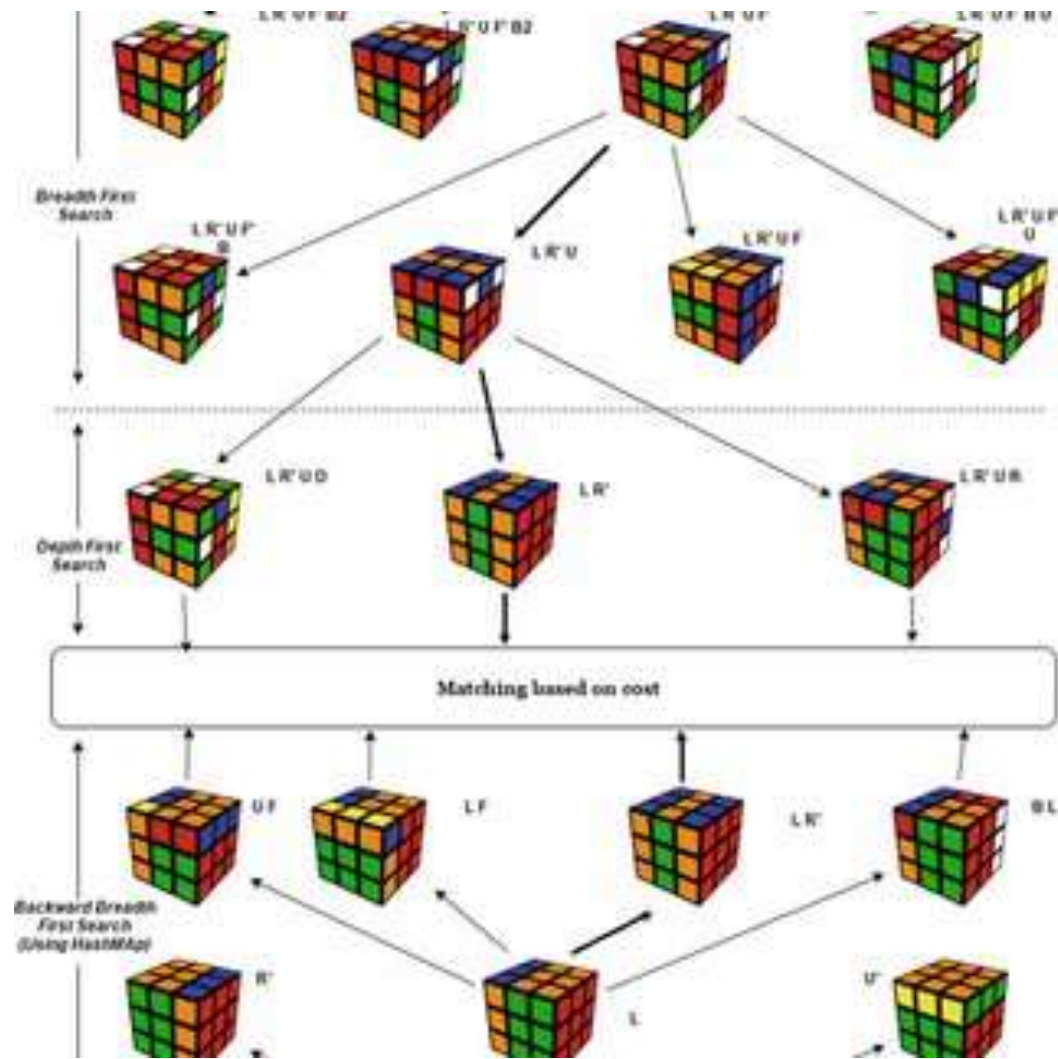


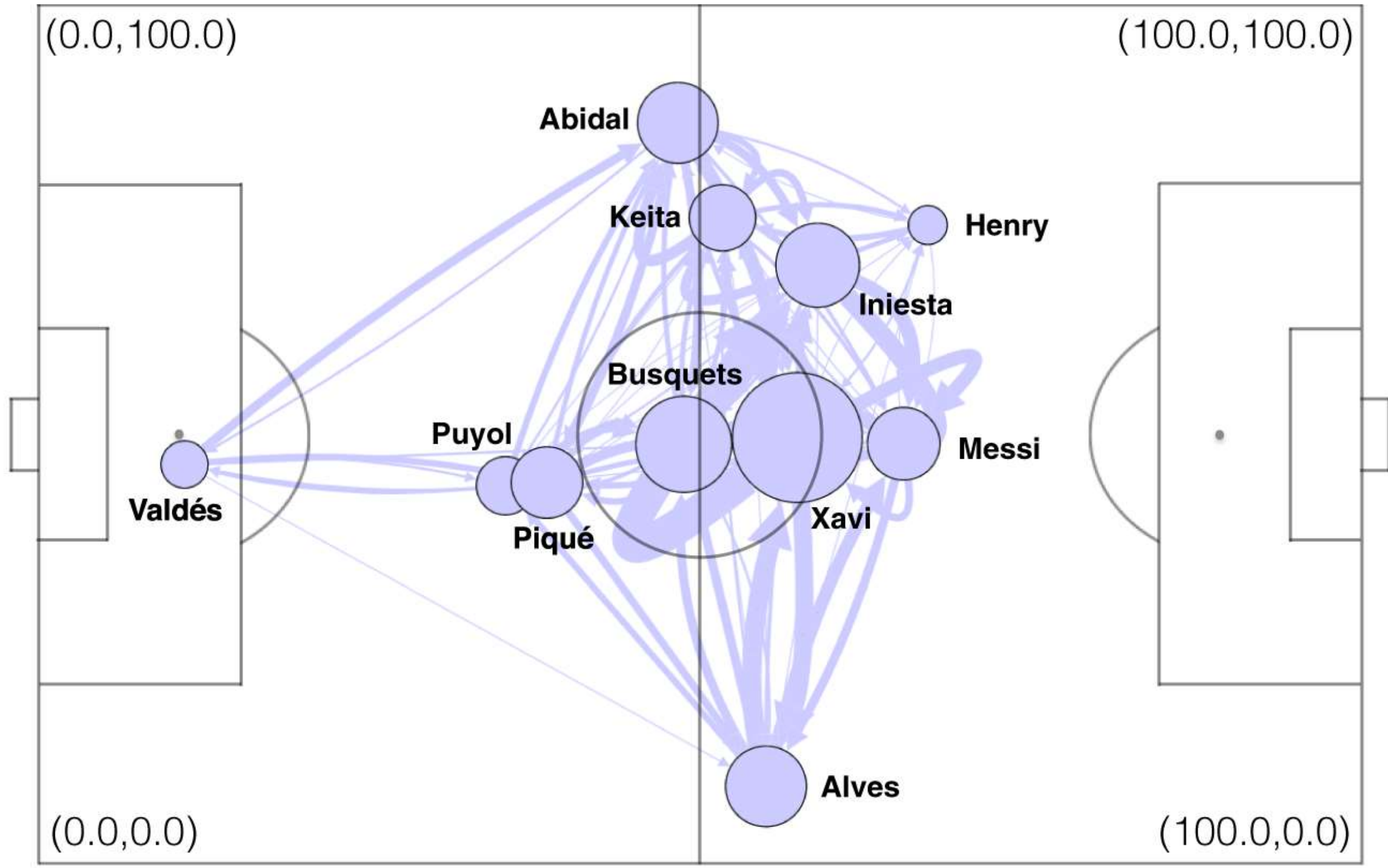
facebook

December 13, 2010

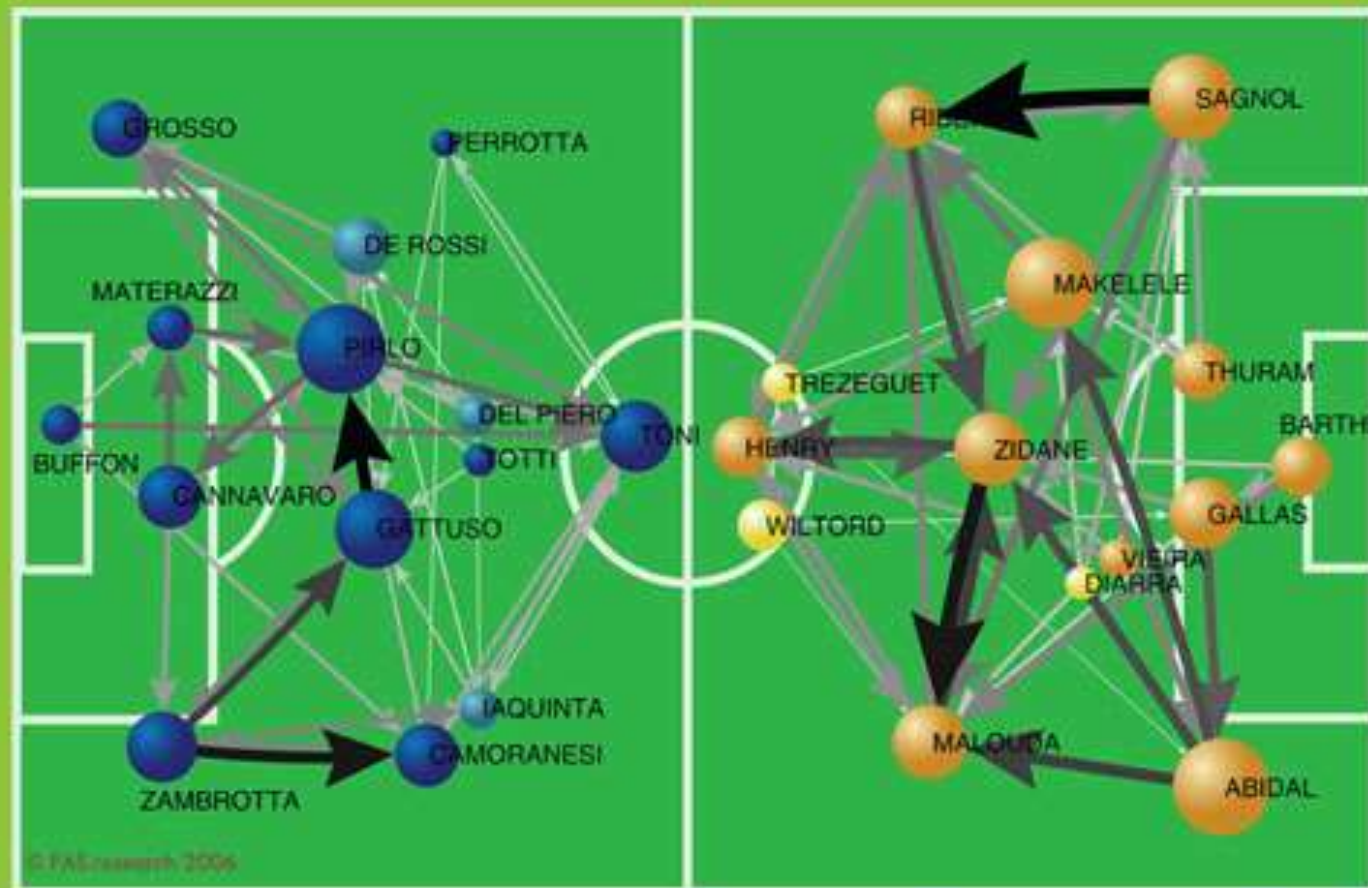




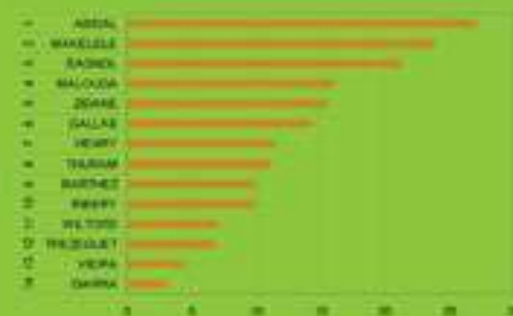




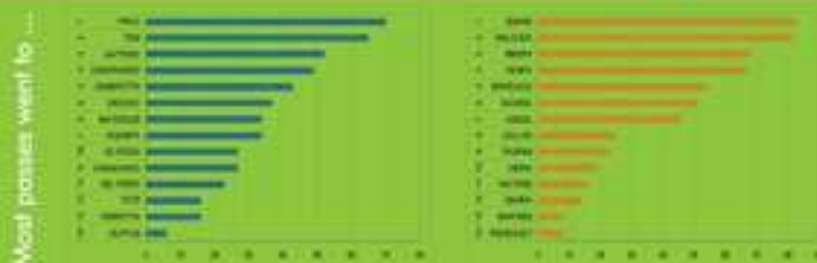
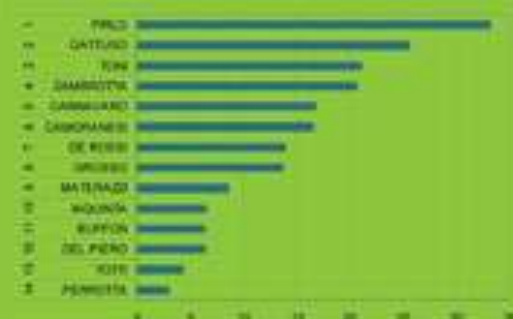
WORLD CUP 2006 *Final*



The most frequent (>15) passes between players



Keyplayer (hub in x % of activities)



This network shows the passes from every player to those three team-mates, he passes to most frequently. Strength of arcs displays the number of passes. Size of nodes displays the influence (flowbetweenness) of a player.

The match was coded and analyzed by Harald Katzmaier and Helmut Neundlinger, Vienna, July 9th, 2006.

FAS.research



Network Analysis for Science and Business

Warschau, 28. 6. 2012

LAHM [0-90]

PODOLSKI [0-45]

SCHWEINSTEIGER [0-90]

ÖZIL [0-90]

GOMEZ [0-45]

KLOSE [45-90]

KROOS [0-90]

MÜLLER [71-90]

REUS [45-90]

BADSTUBER [0-90]

NEUER [0-90]

HUMMELS [0-90]

KHEDIRA [0-90]

BOATENG [0-71]

© FAS.research



DEUTSCHLAND – ITALIEN 1:2

MARCHISIO [0-90]

BALZARETTI [0-90]

BARZAGLI [0-90]

PIRLO [0-90]

MONTOLIVO [0-63]

BALOTELLI [0-70]

DI NATALE [70-90]

MOTTA [63-90]

DE ROSSI [0-90]

CASSANO [0-57]

BUFFON [0-90]

BONUCCI [0-90]

CHIELLINI [0-90]

[*] Einsatzminuten

DER STANDARD

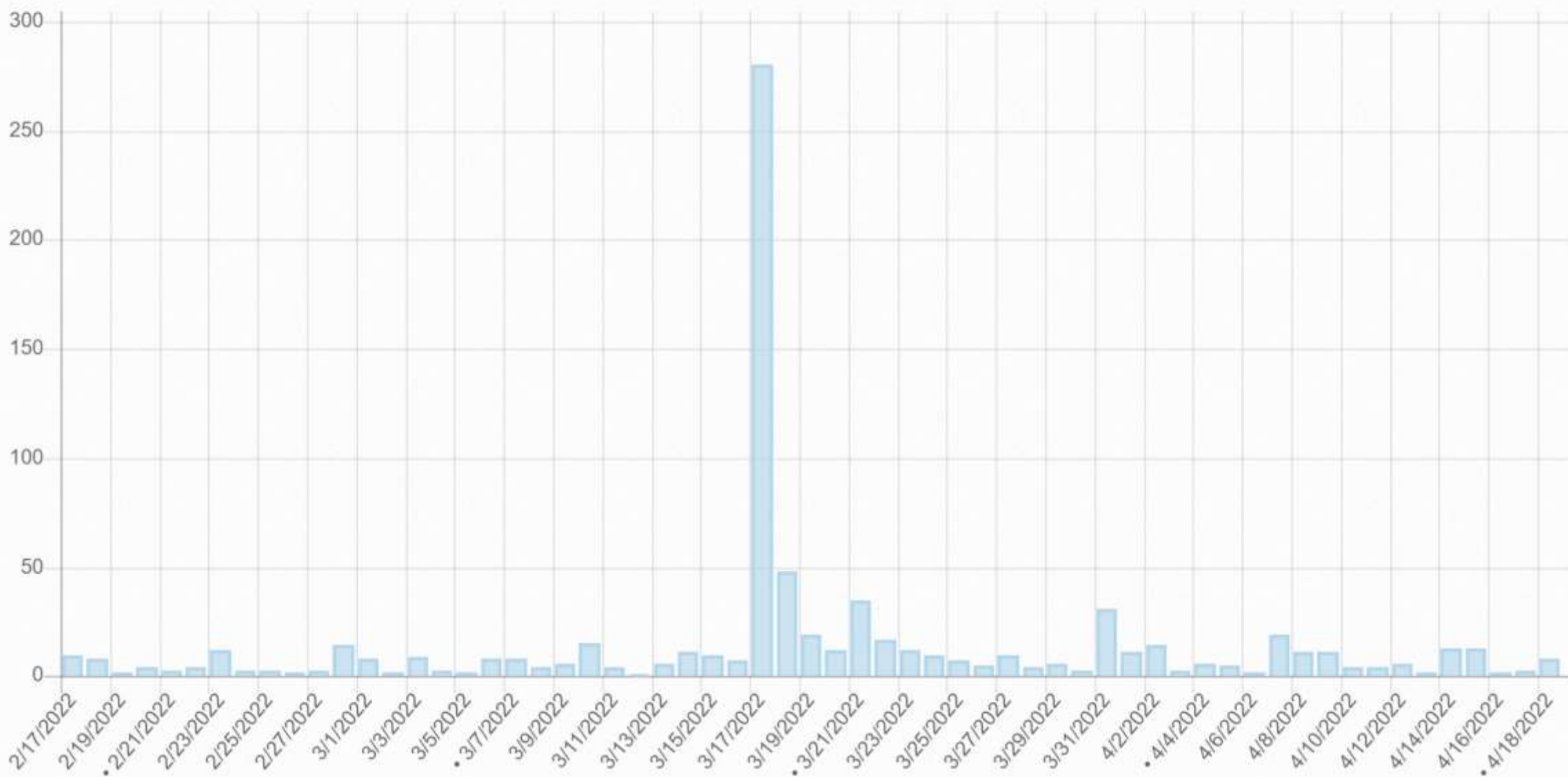
✕ Giuseppe_F._Italiano

Chart type

Permalink

Download ▾

☐ Show values ☐ Begin at zero ☐ Logarithmic scale



ORA CRACCO LA PAGINA SU
WIKIPEDIA DEL PROF. ITALIANO



QUESTO NON SI FA!!!



GUARDIANO DI WIKIPEDIA

Giuseppe F. Italiano: Revision history

[View logs for this page](#) ([view filter log](#))

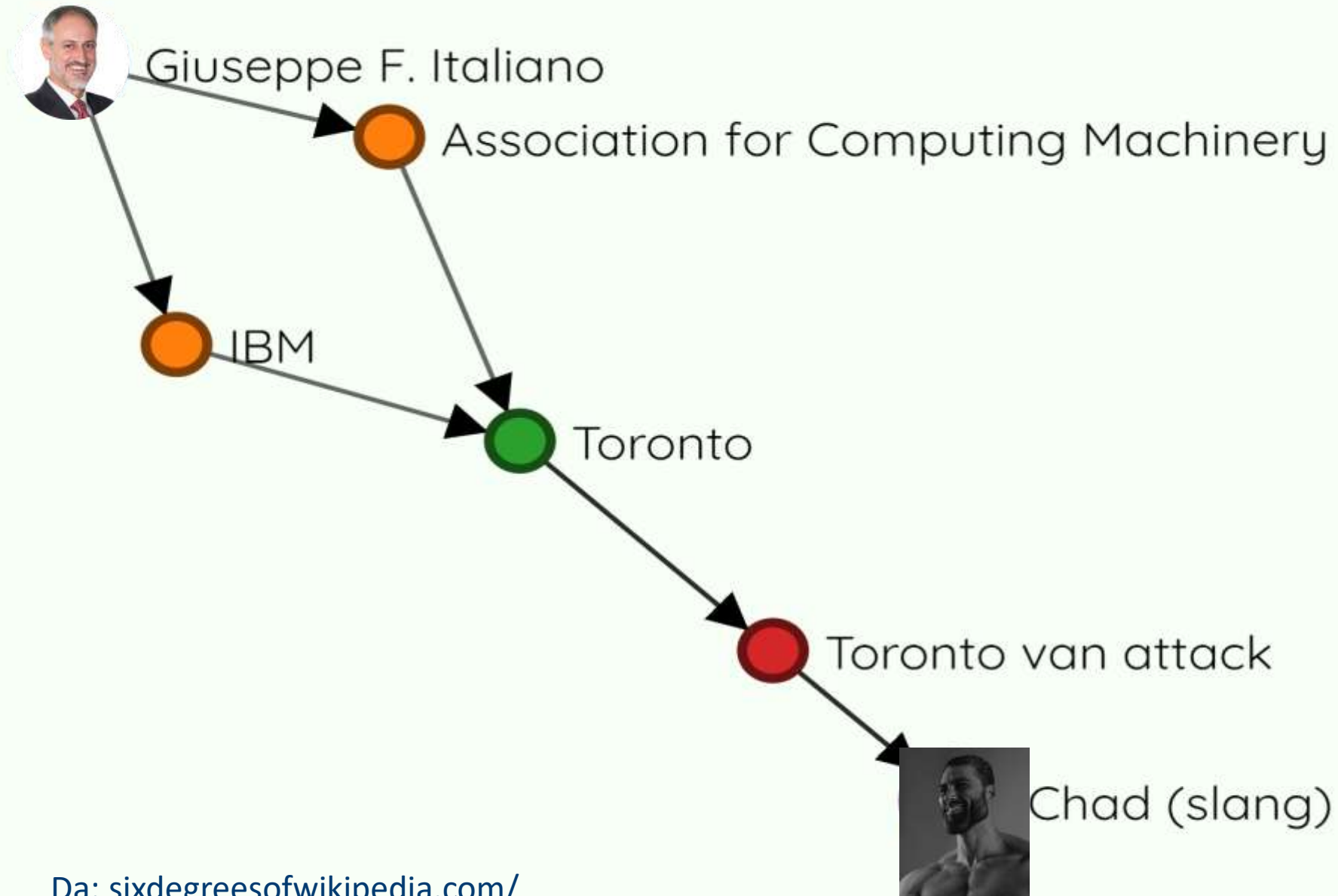
▼ **Filter revisions**

External tools: [Find addition/removal](#) ([Alternate](#)) · [Find edits by user](#) ([Alternate](#)) · [Page statistics](#) · [Pageviews](#) · [Fix dead links](#)

For any version listed below, click on its date to view it. For more help, see [Help:Page history](#) and [Help:Edit summary](#). (cur) = difference from current version, (prev) = difference from preceding version, **m** = [minor edit](#), → = [section edit](#), ← = [automatic edit summary](#) (newest | [oldest](#)) View (newer 50 | [older 50](#)) ([20](#) | [50](#) | [100](#) | [250](#) | [500](#))

Compare selected revisions

- (cur | prev) ☒ 02:20, 25 March 2022 MusikBot II (talk | contribs) **m** . . (5,746 bytes) (−17) . . (Removing protection templates from unprotected page ([more info](#))) (undo) (Tag: Manual revert)
- (cur | prev) ☒ 19:00, 18 March 2022 MusikBot II (talk | contribs) **m** . . (5,763 bytes) (+17) . . (Adding missing protection template ([more info](#))) (undo) (Tag: Reverted)
- (cur | prev) ☐ 18:54, 18 March 2022 David Eppstein (talk | contribs) **m** . . (5,746 bytes) (0) . . (Protected "[Giuseppe F. Italiano](#)": Violations of the [biographies of living persons policy](#) ([Edit=Require autoconfirmed or confirmed access] (expires 18:54, 21 March 2022 (UTC))) [Move=Require autoconfirmed or confirmed access] (expires 18:54, 21 March 2022 (UTC)))) (undo)
- (cur | prev) ☐ 18:54, 18 March 2022 David Eppstein (talk | contribs) **m** . . (5,746 bytes) (−12) . . (Reverted edits by [151.43.240.143](#) ([talk](#)) to last version by David Eppstein) (undo) (Tag: Rollback)
- (cur | prev) ☐ 08:07, 18 March 2022 [151.43.240.143](#) ([talk](#)) . . (5,758 bytes) (+12) . . (undo) (Tags: Reverted, Mobile edit, Mobile web edit)
- (cur | prev) ☐ 17:43, 17 March 2022 David Eppstein (talk | contribs) **m** . . (5,746 bytes) (−16) . . (Reverted edits by [SimonePellegatta](#) ([talk](#)) to last version by [91.125.209.214](#)) (undo) (Tag: Rollback)
- (cur | prev) ☐ 17:37, 17 March 2022 [SimonePellegatta](#) (talk | contribs) **m** . . (5,762 bytes) (+16) . . (undo) (Tag: Reverted)

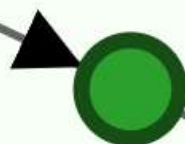




Chad (slang)



International Standard Book Number

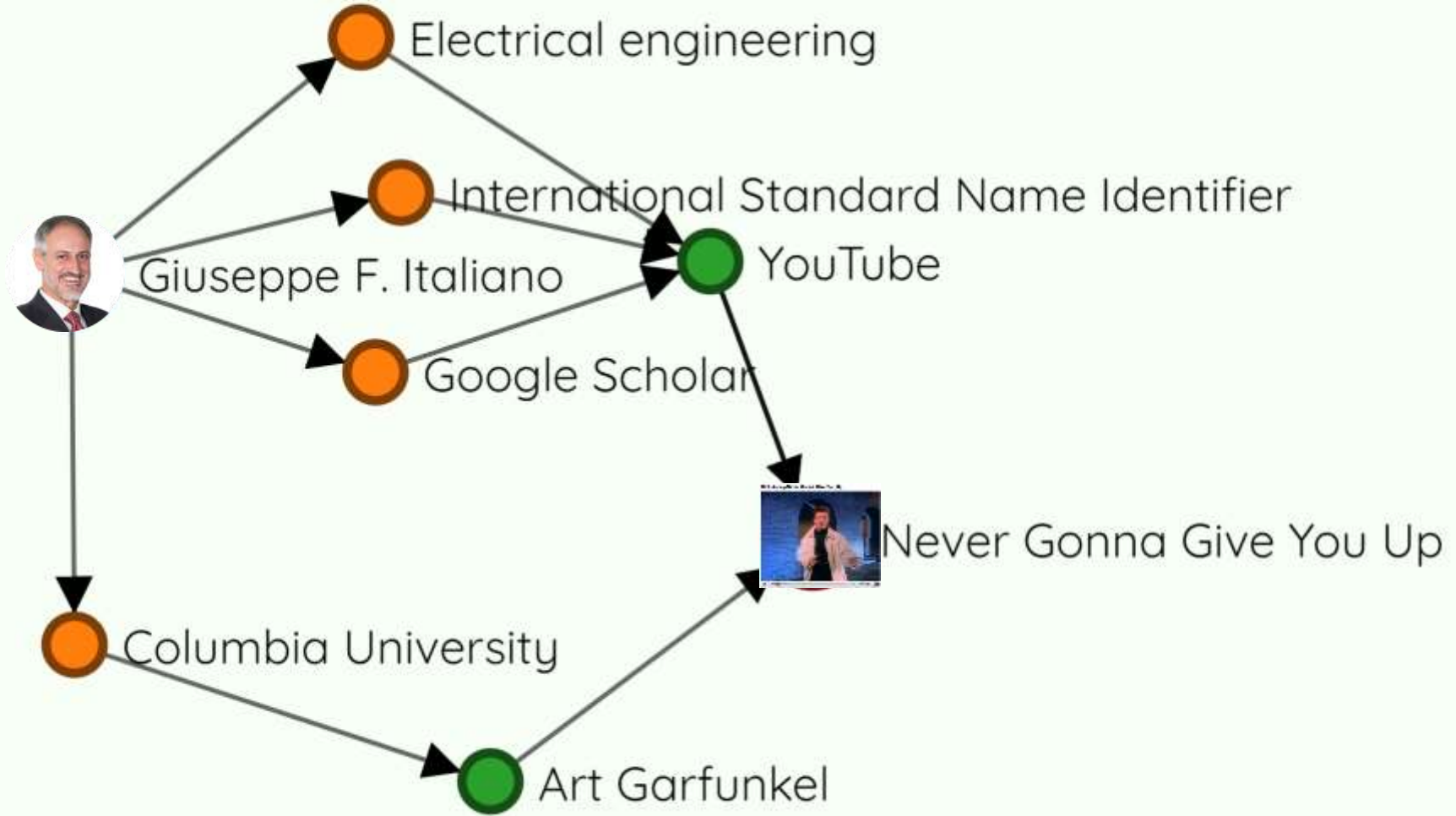


QR code



Giuseppe F. Italiano

Da: sixdegreesofwikipedia.com/



Da: sixdegreesofwikipedia.com/

Definizione

Un grafo $G = (V, E)$ consiste in:

- un insieme V di nodi (o nodi)
- un insieme E di coppie di nodi, detti archi: ogni arco connette due nodi

Esempio 1 Grafo di Facebook: $V = \{\text{utenti Facebook}\}$,
 $E = \{\text{amicizie su Facebook}\}$

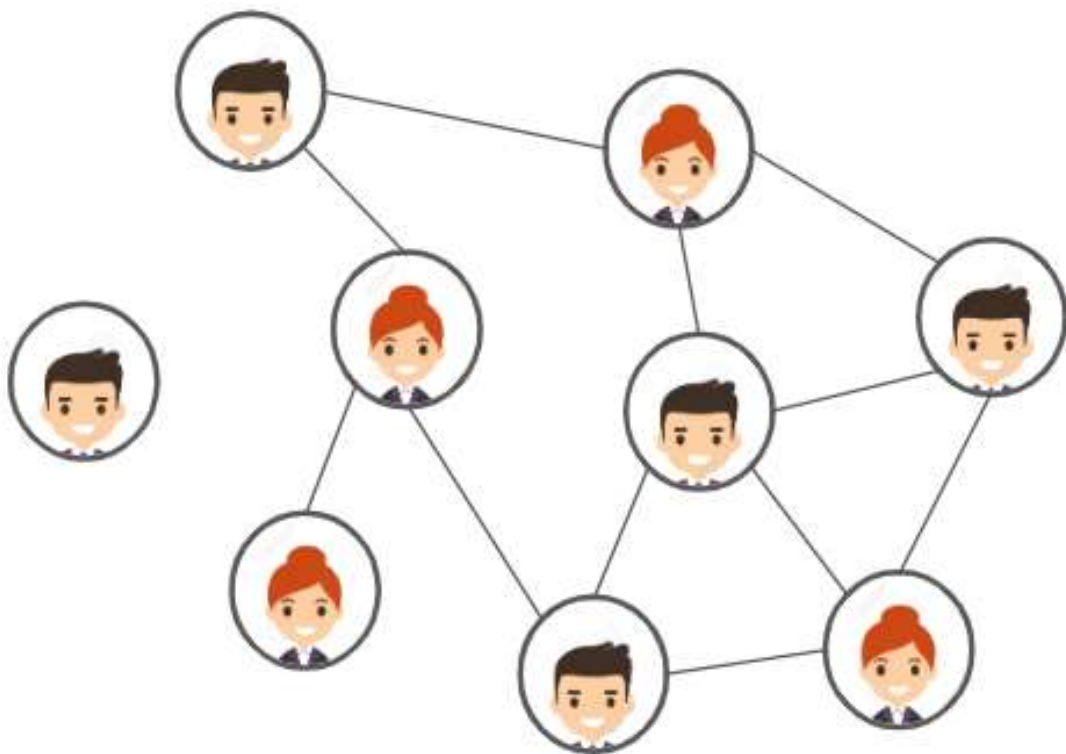


Esempio 2 Grafo di Instagram: $V = \{\text{utenti Instagram}\}$,
 $E = \{(x, y) \text{ tale che } x \text{ segue } y \text{ su Instagram}\}$

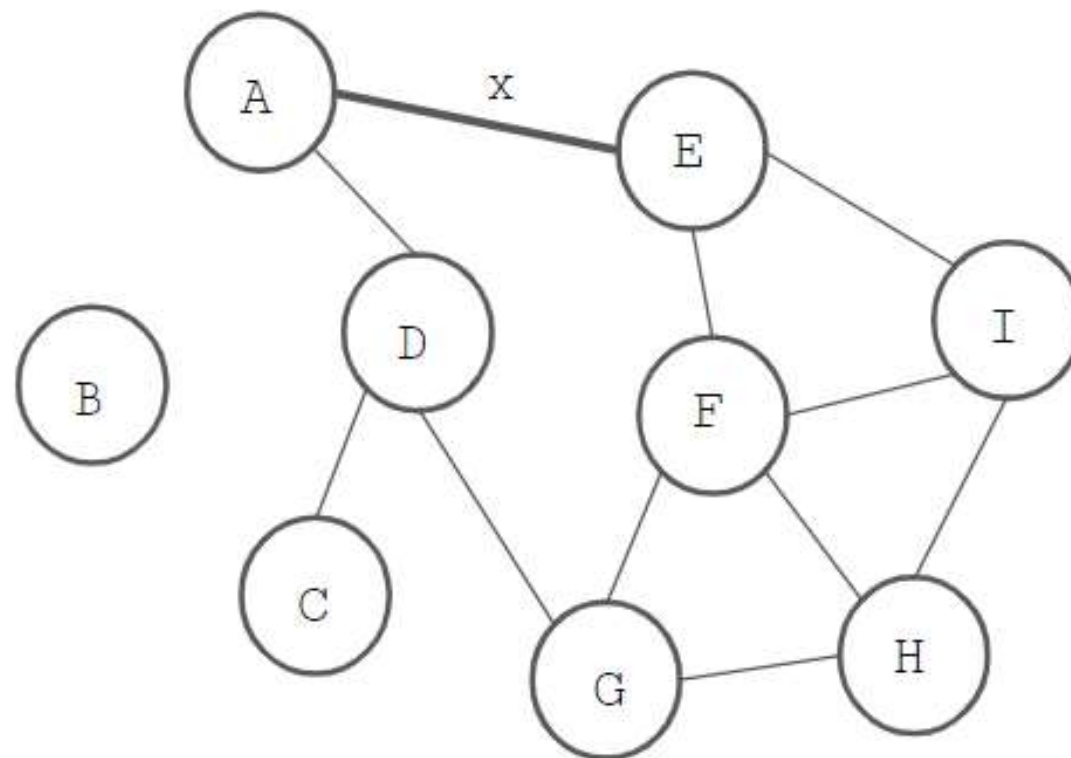


Grafi

Social Network

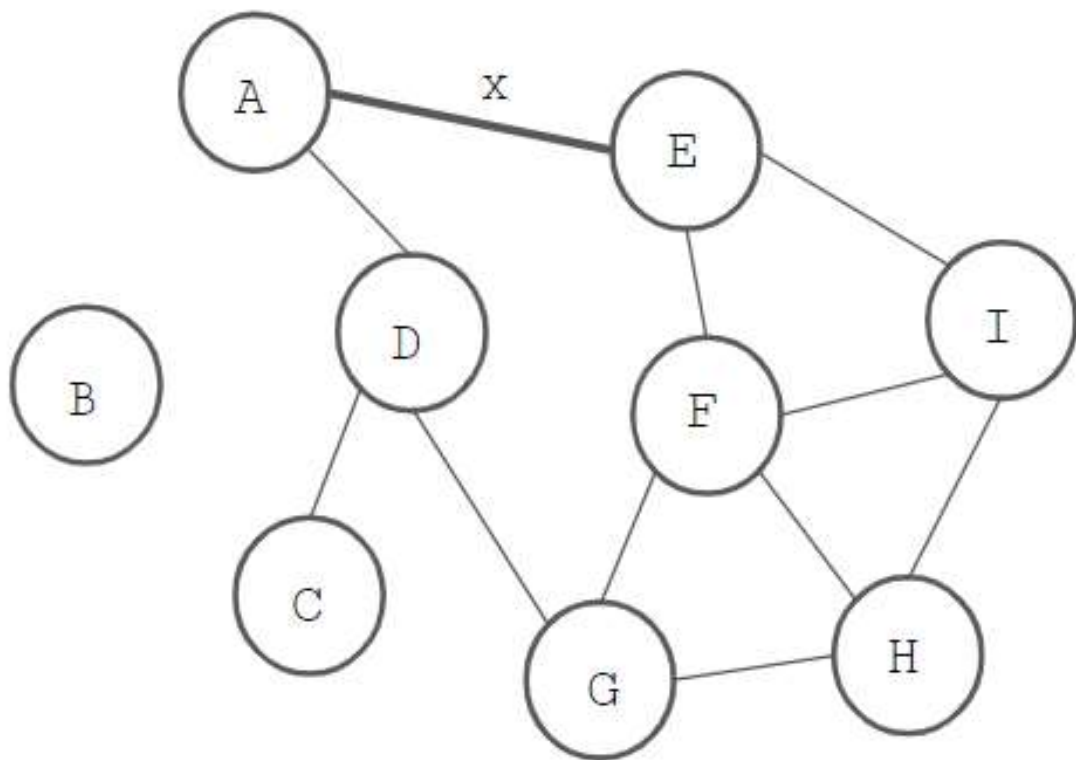


$G = (V, E)$



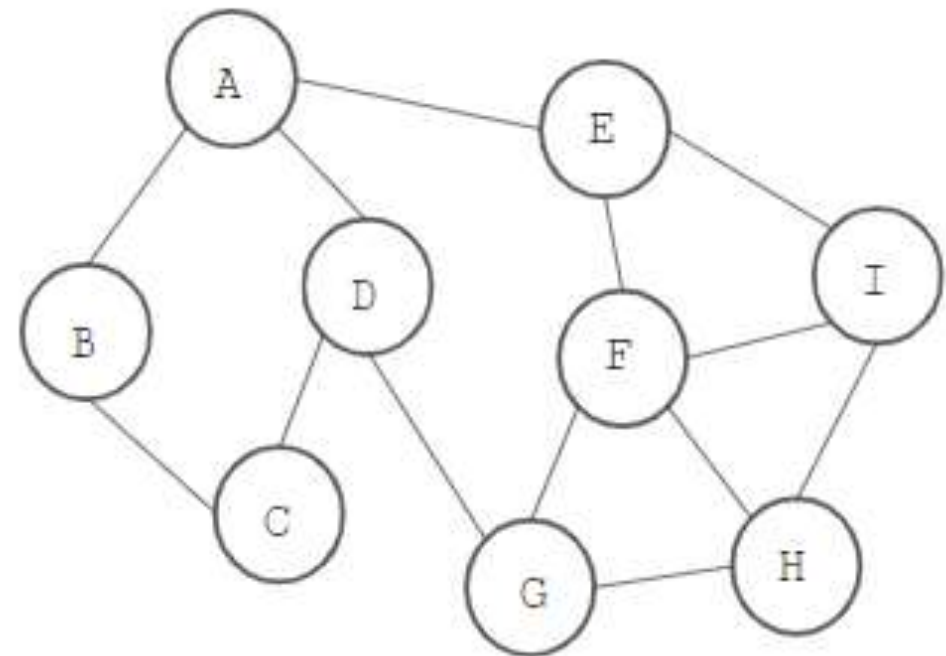
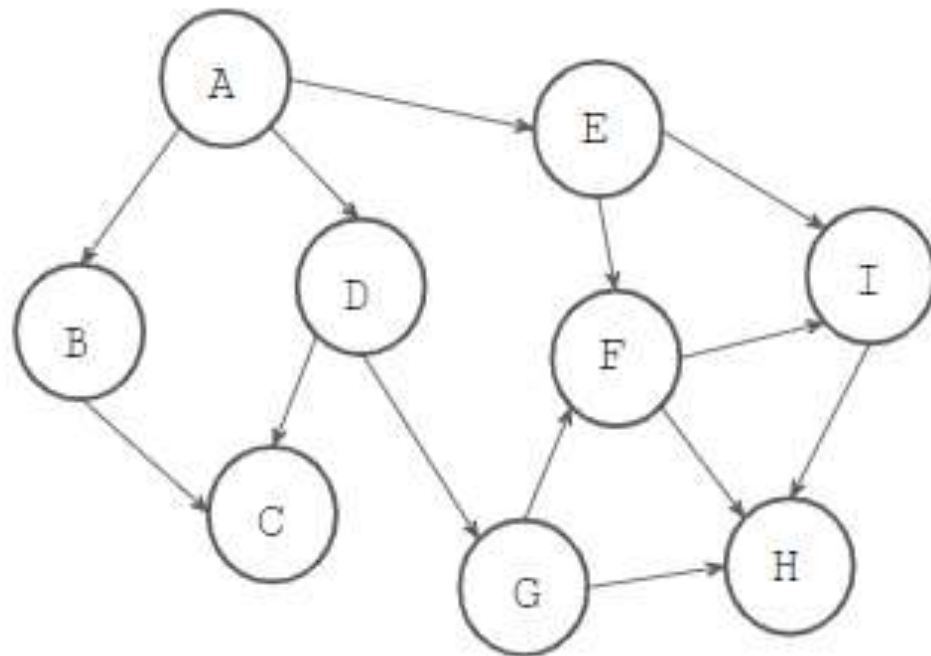
Grafi

$$G = (V, E)$$

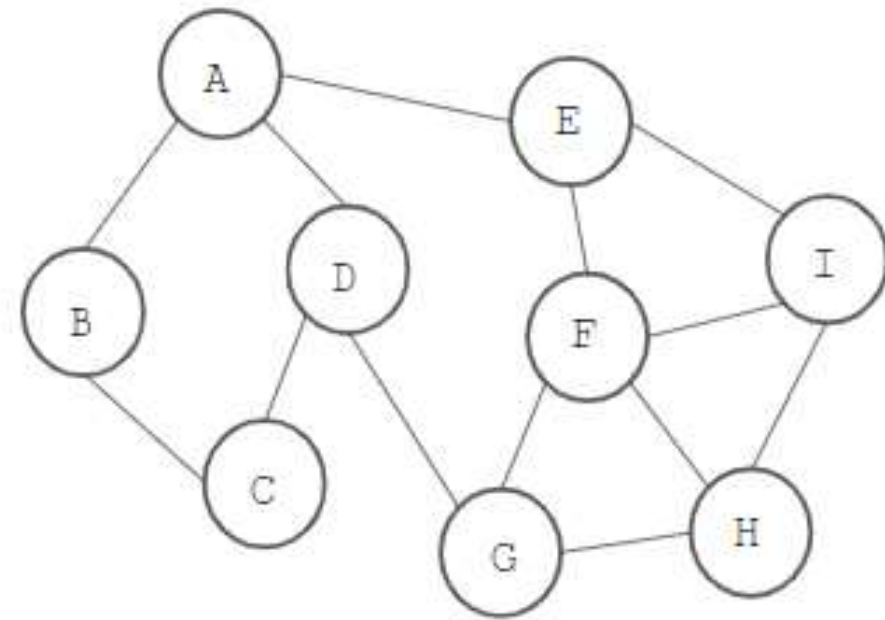
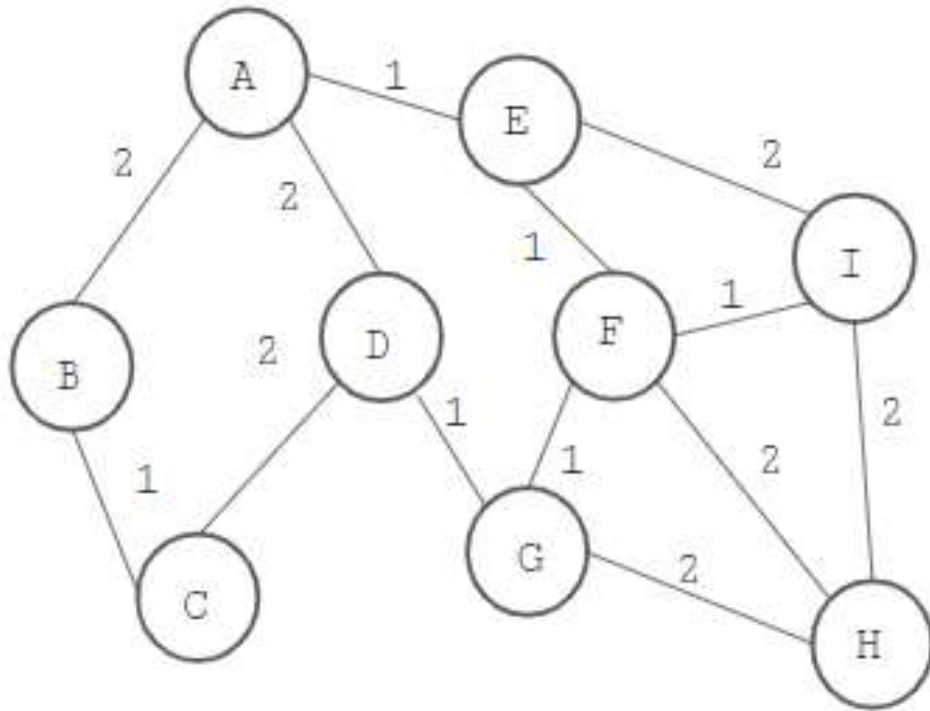


- L'arco x è **incidente** sui nodi A ed E (ogni arco è incidente su due nodi)
- Il nodo G è **adiacente** ai nodi D, F, H (nodi adiacenti sono connessi da un arco)
- Il nodo B è **isolato**
- A ha **grado** 2, B ha grado 0, F ha grado 4 (numero di archi incidenti)
- C'è un **cammino** (di lunghezza 3) tra A e H (A, E, I, H).
- Il cammino (E, F, I, E) è un **ciclo** (cammino che comincia e finisce nello stesso nodo)

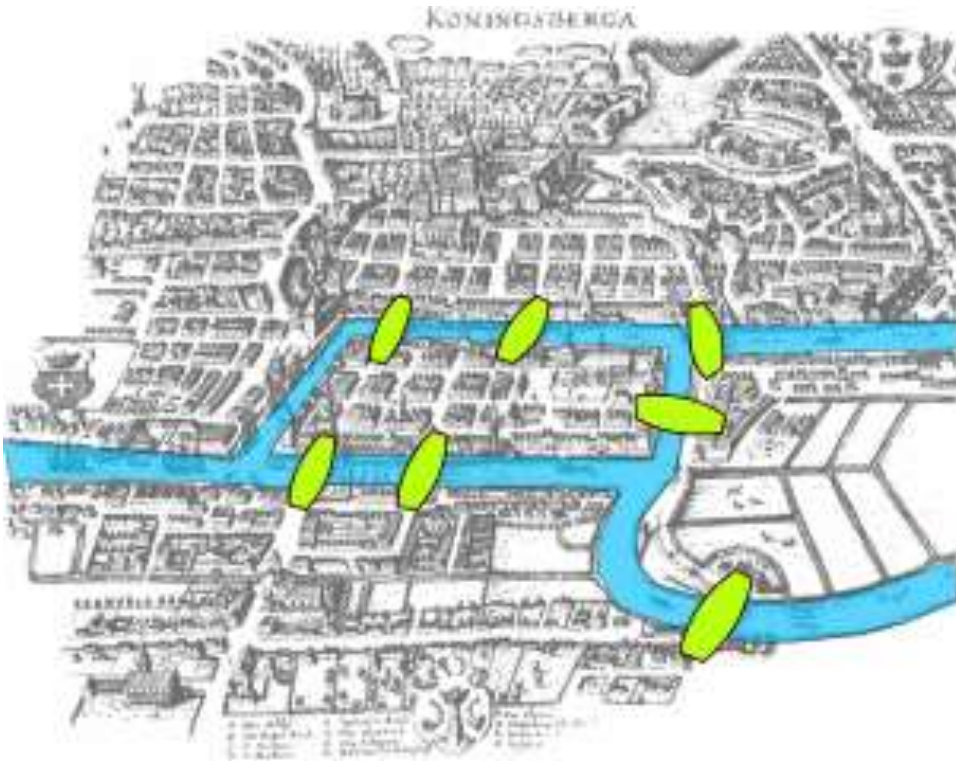
Grafi orientati e non orientati



Grafi pesati e non pesati

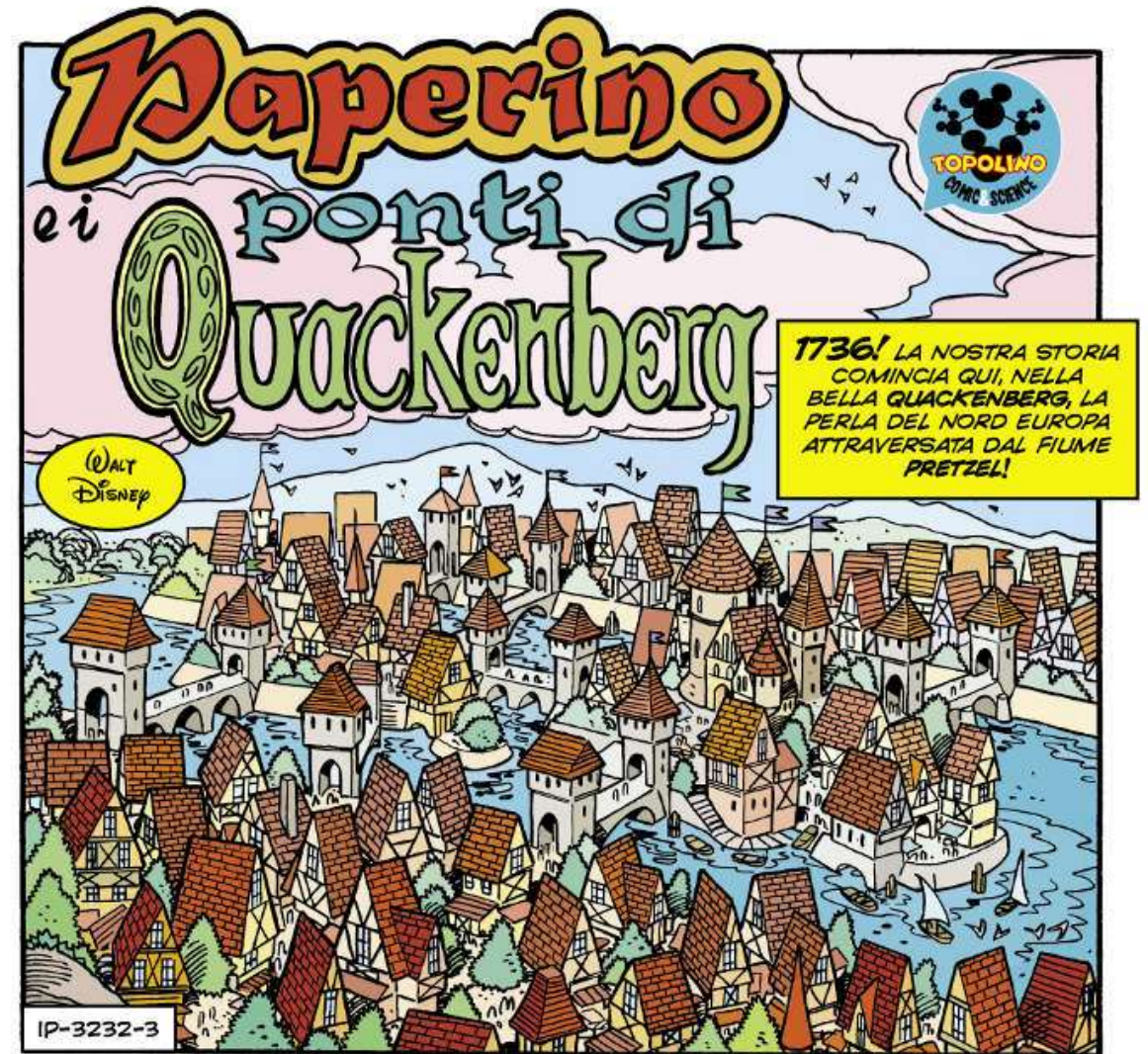
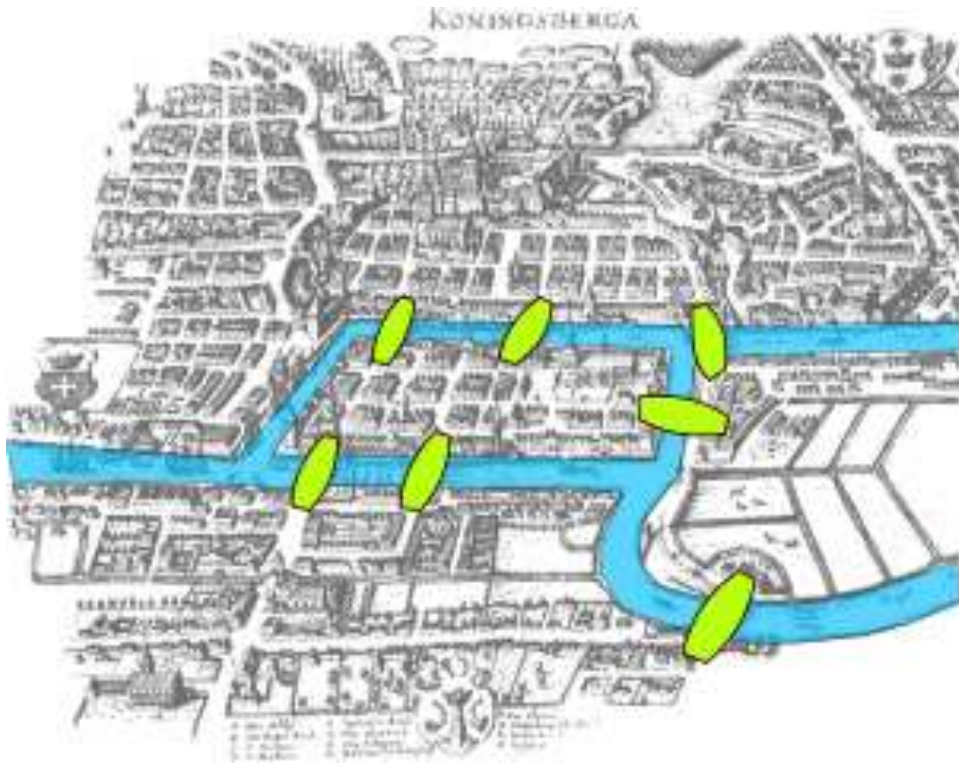


I sette ponti di Königsberg

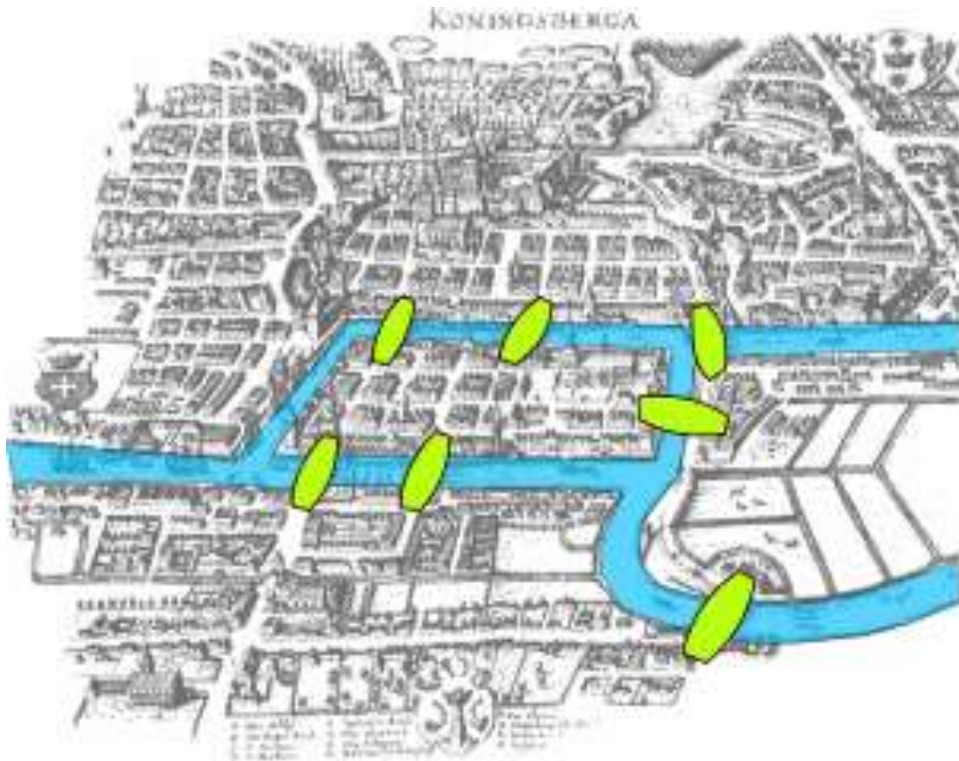


E' possibile con una passeggiata seguire un percorso che attraversi ogni ponte una e una volta soltanto?

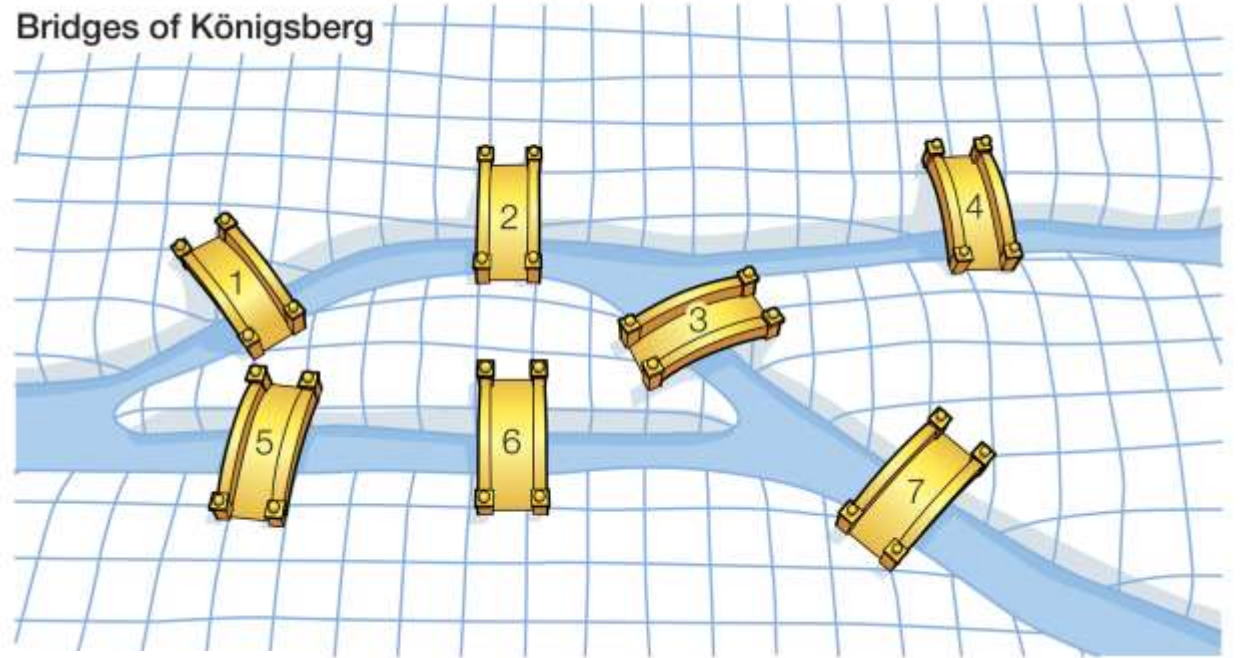
I sette ponti di Königsberg



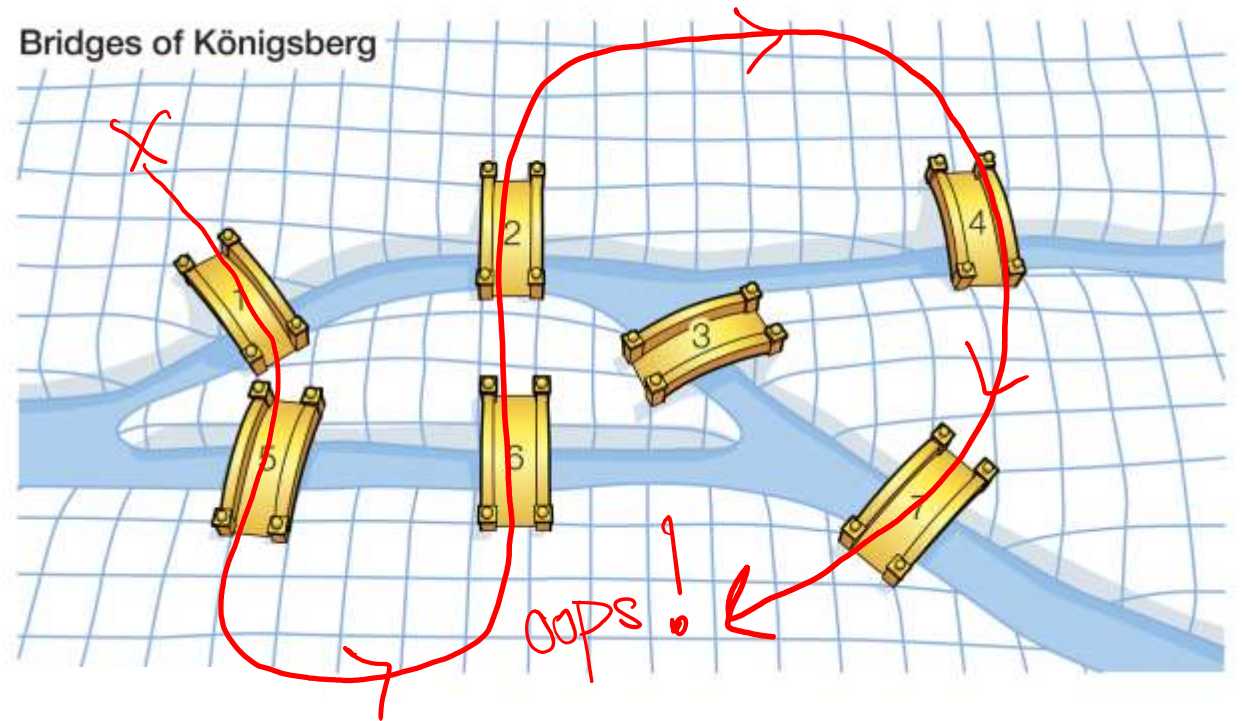
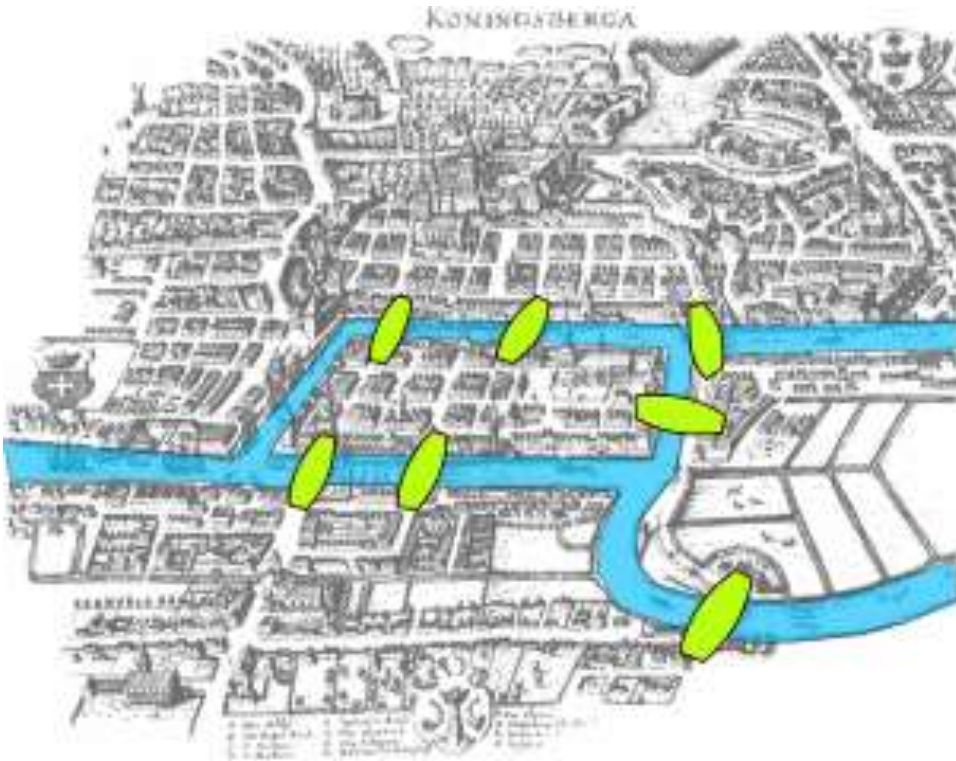
I sette ponti di Königsberg



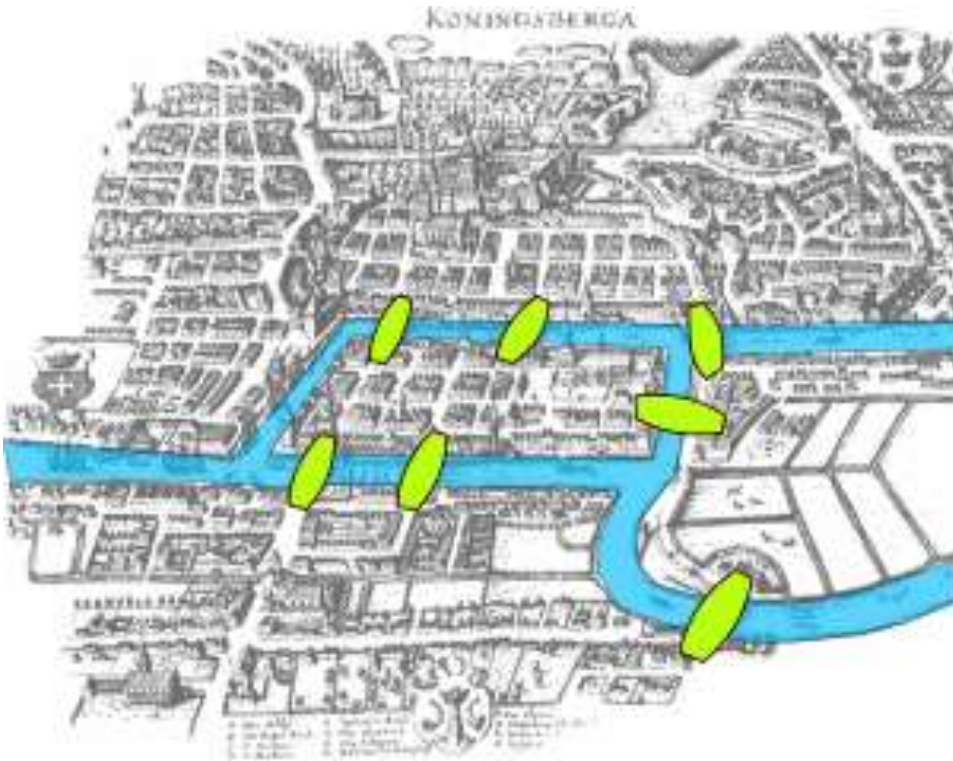
Bridges of Königsberg



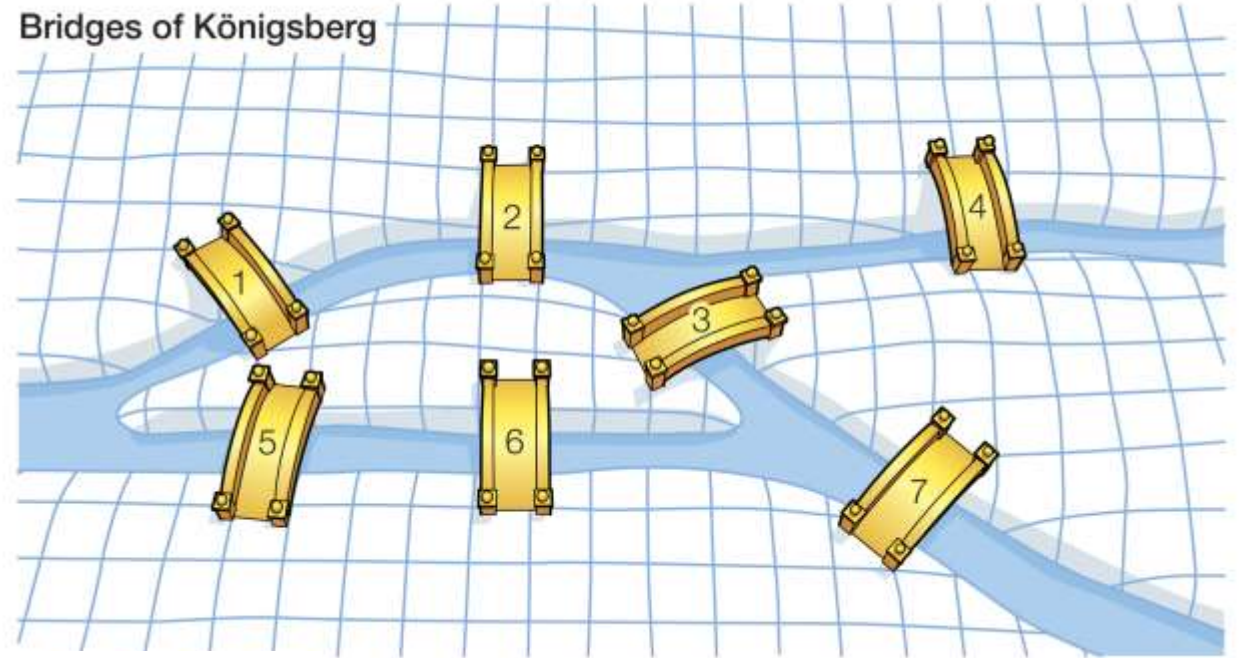
I sette ponti di Königsberg



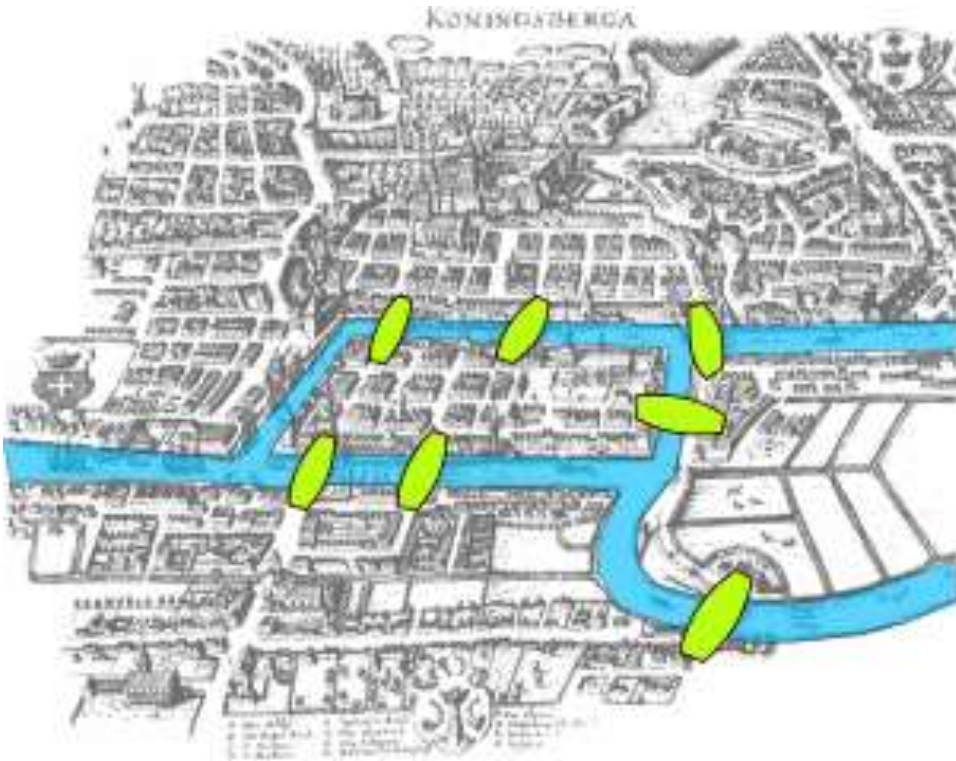
I sette ponti di Königsberg



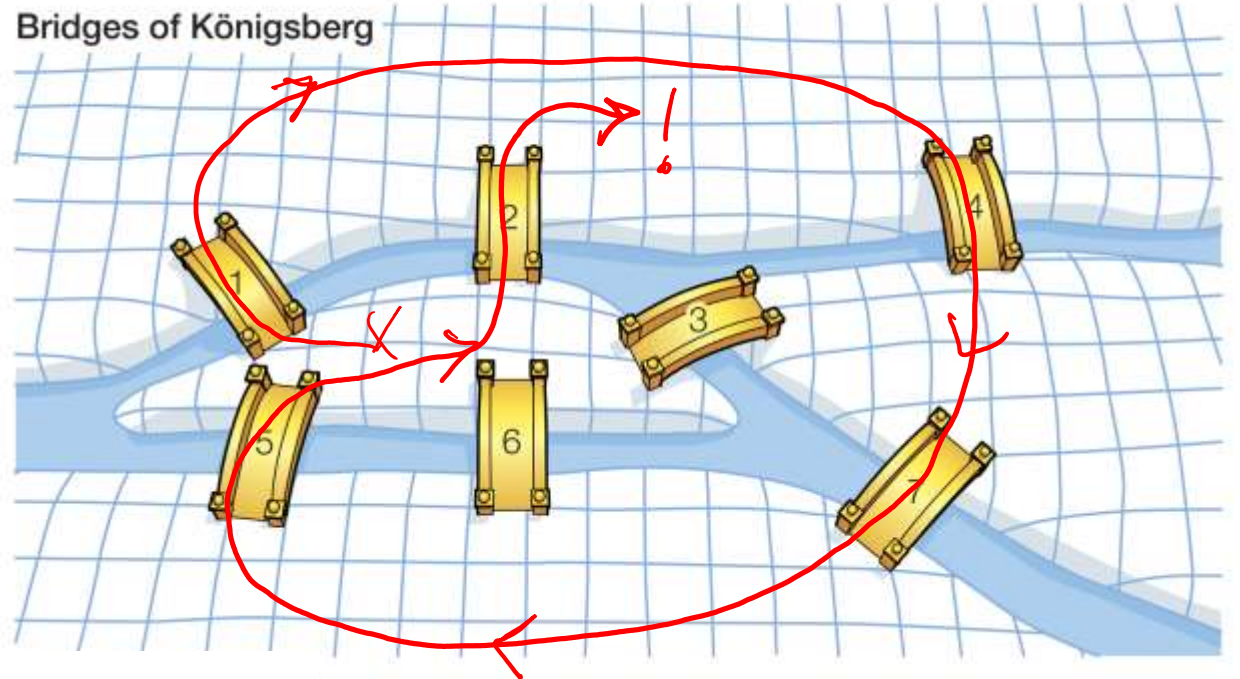
Bridges of Königsberg



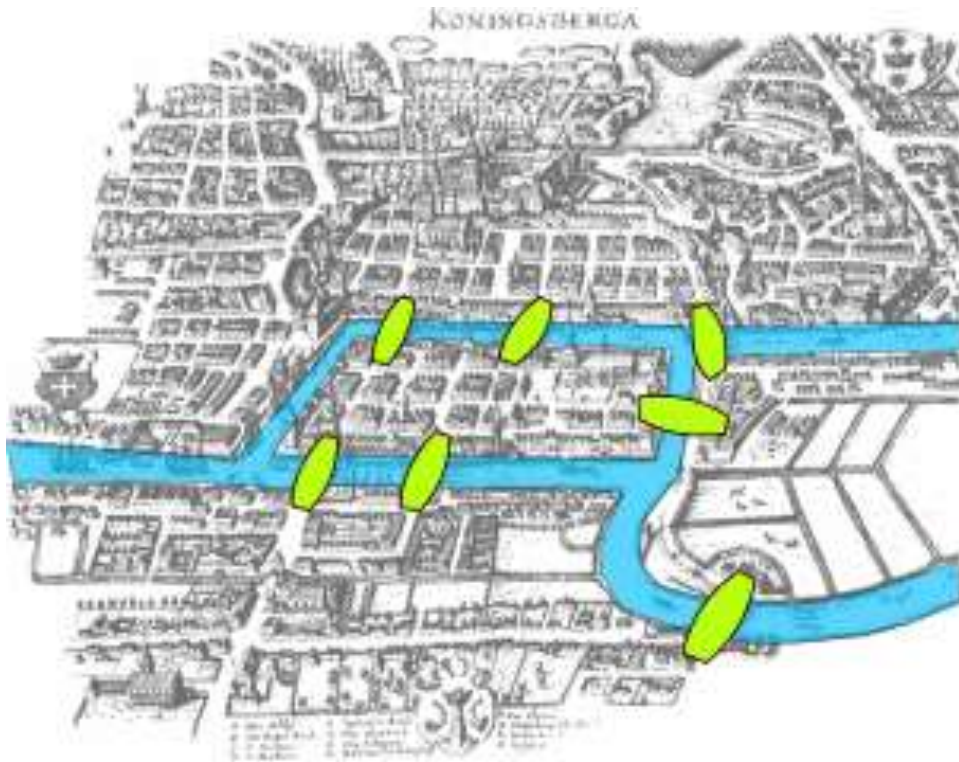
I sette ponti di Königsberg



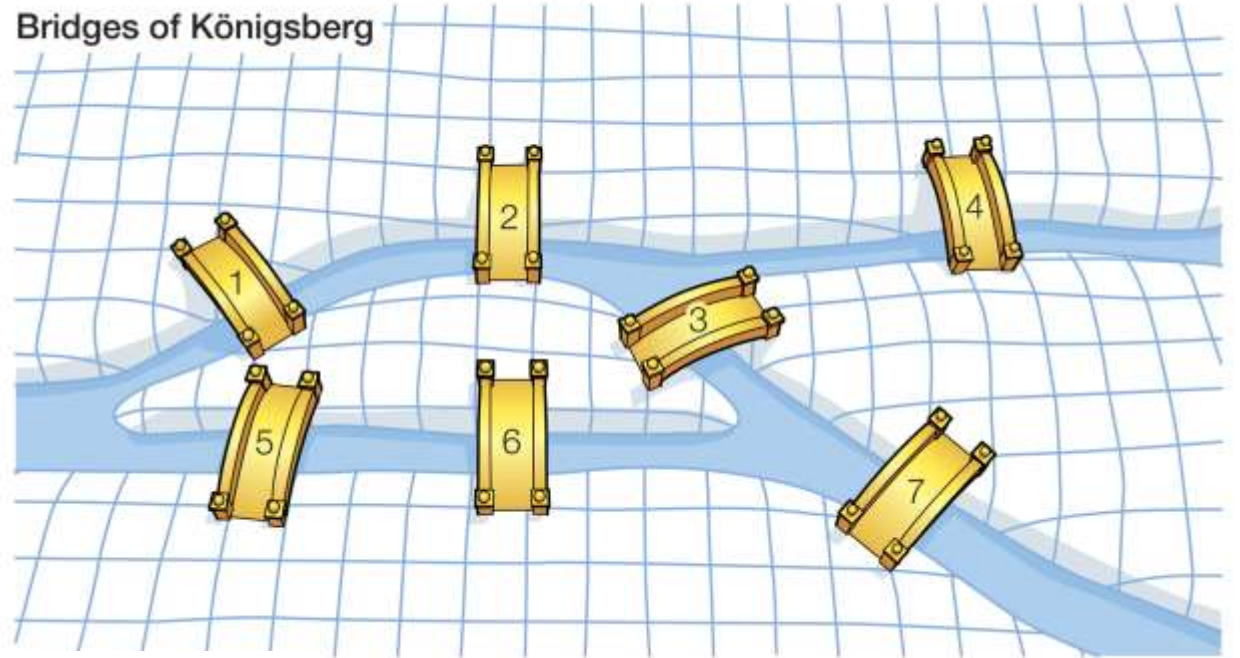
Bridges of Königsberg



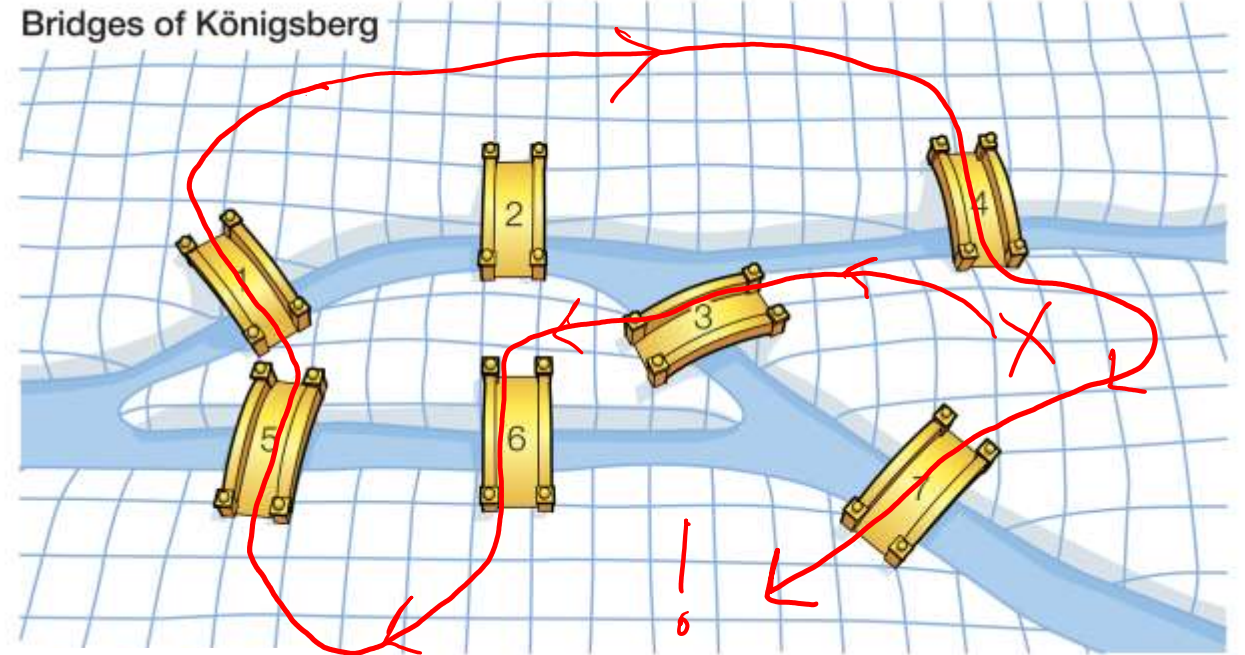
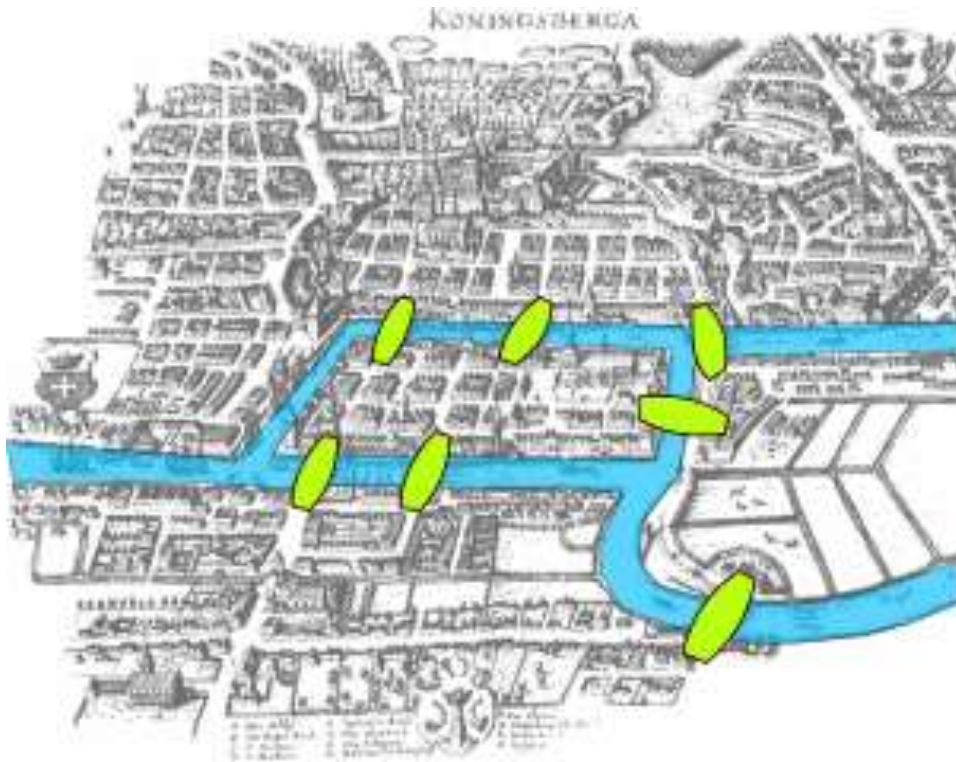
I sette ponti di Königsberg



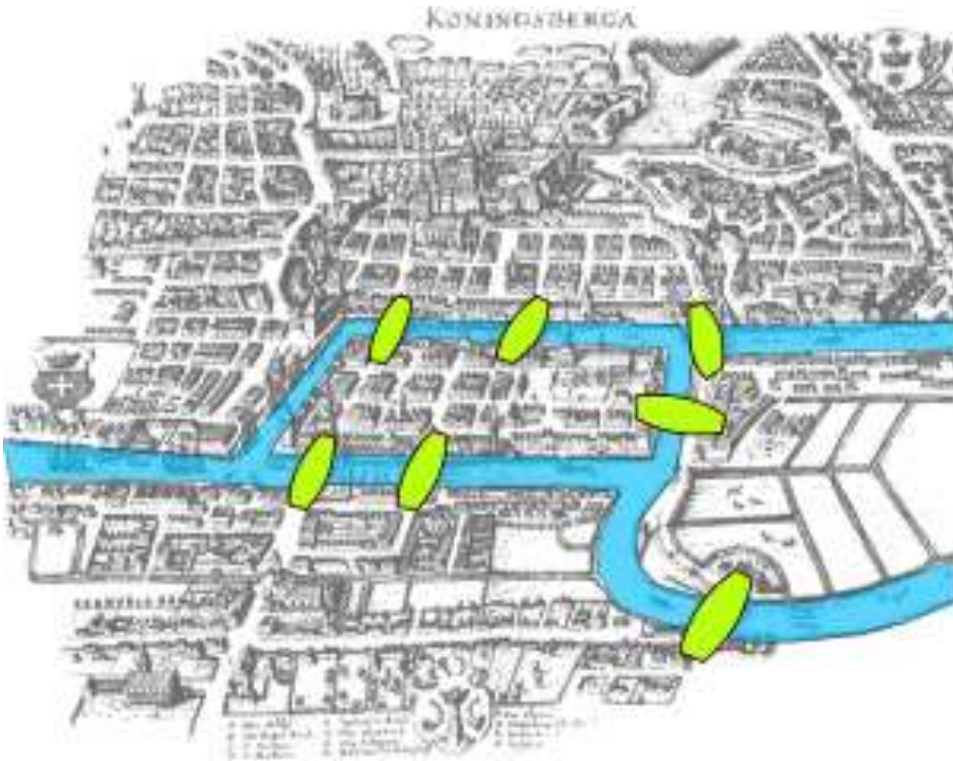
Bridges of Königsberg



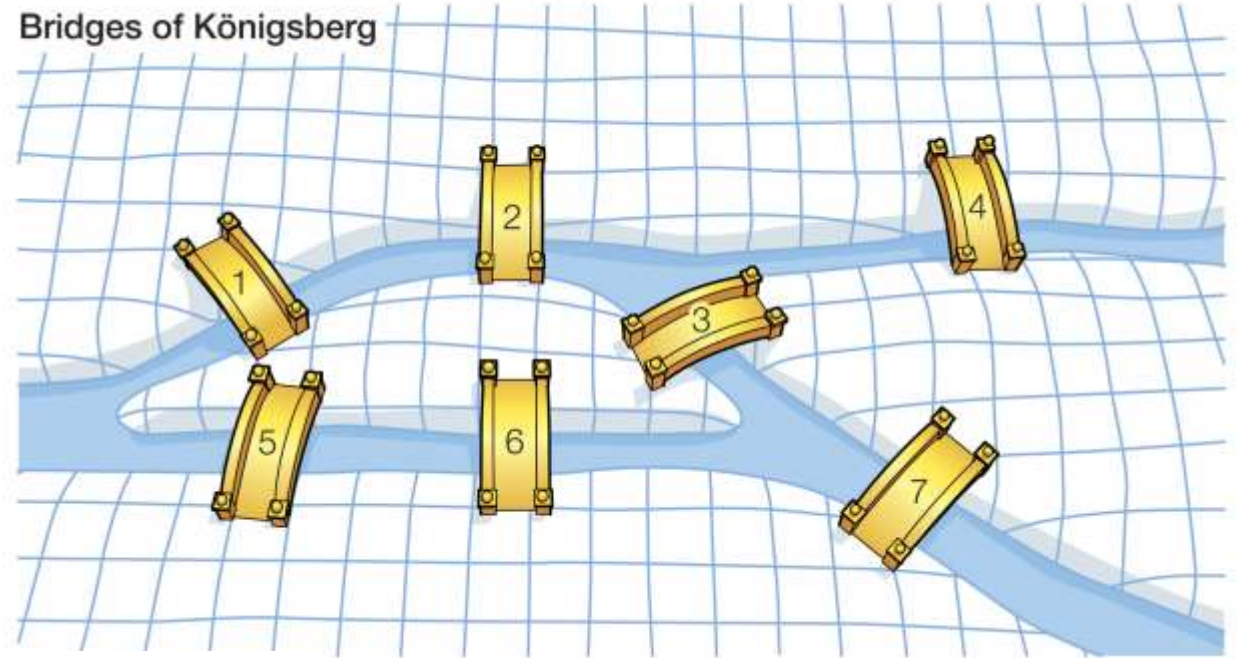
I sette ponti di Königsberg



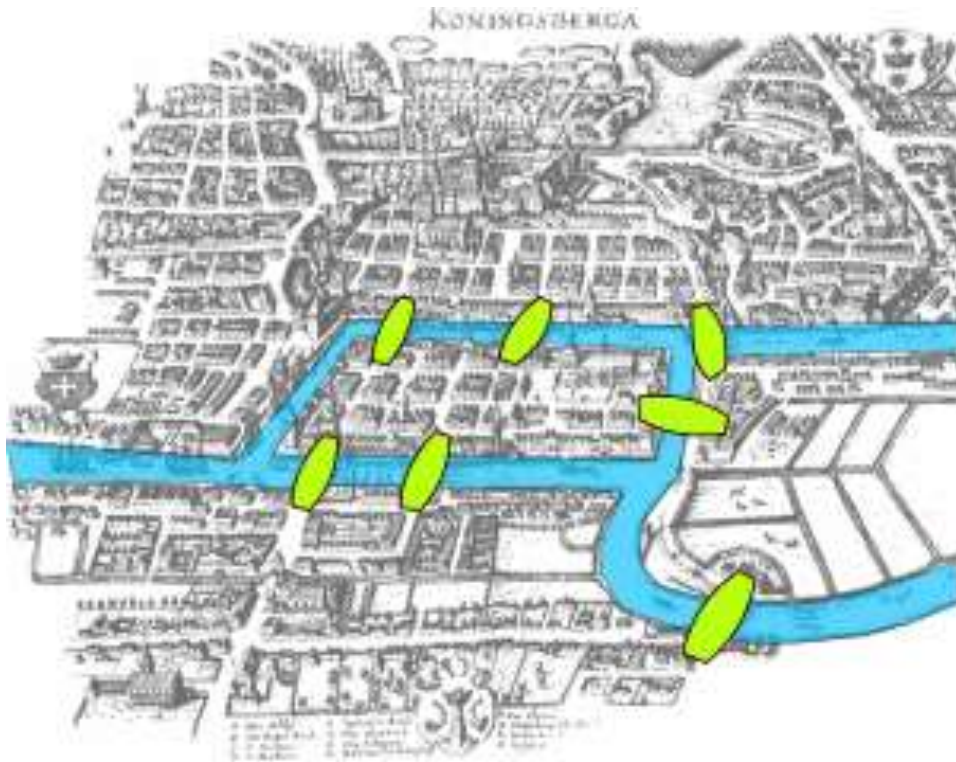
I sette ponti di Königsberg



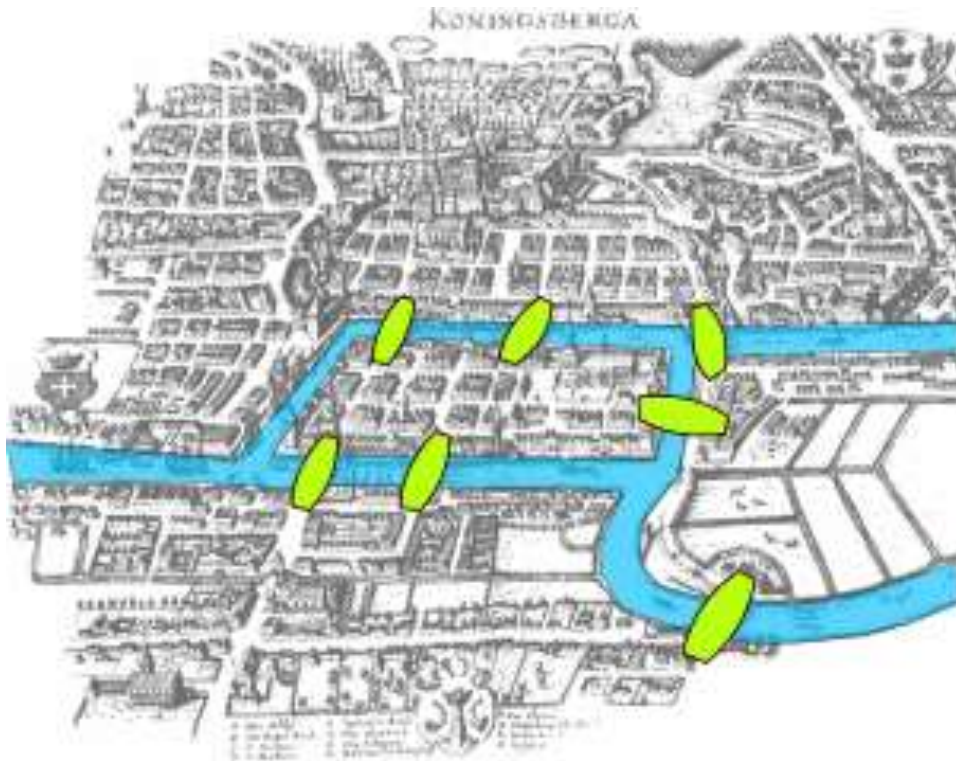
Bridges of Königsberg



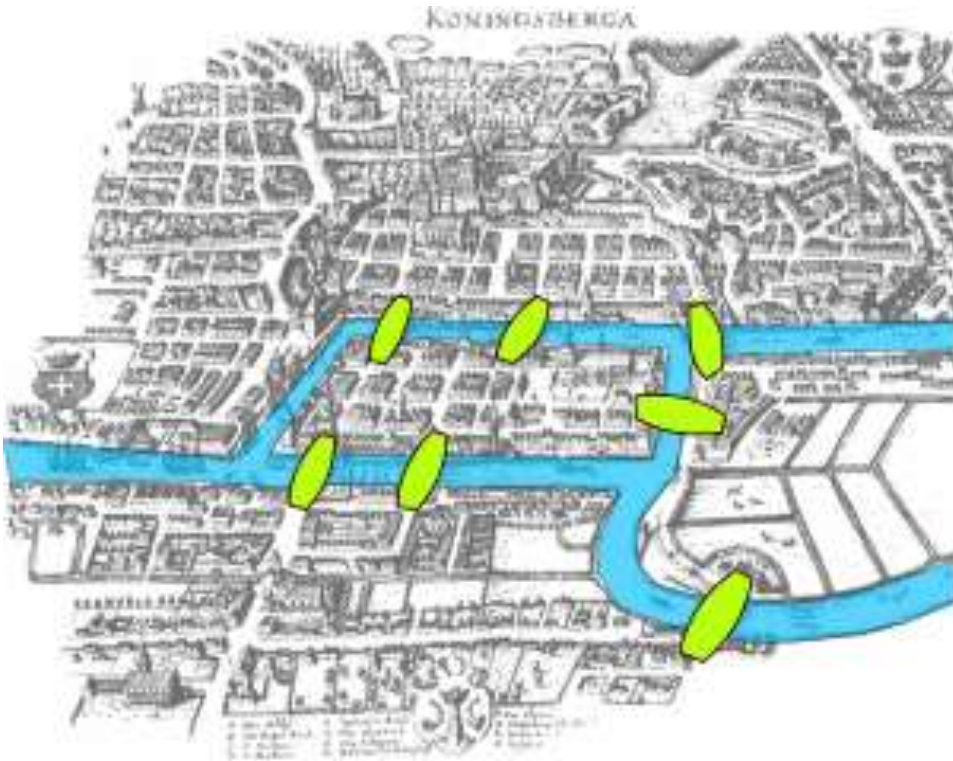
I sette ponti di Königsberg



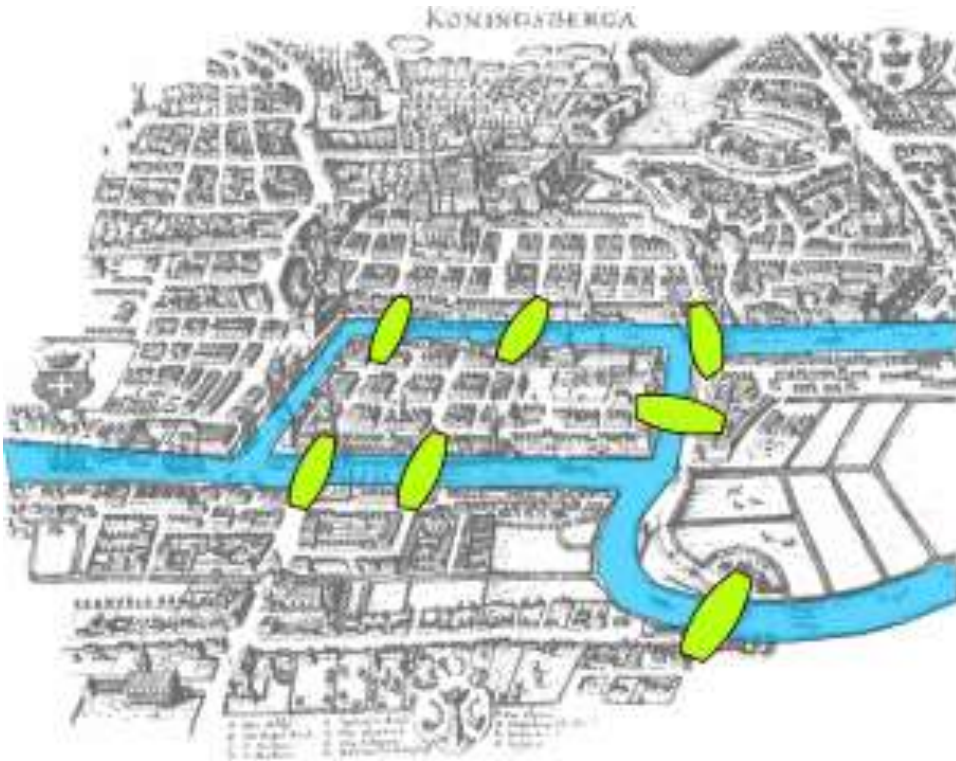
I sette ponti di Königsberg



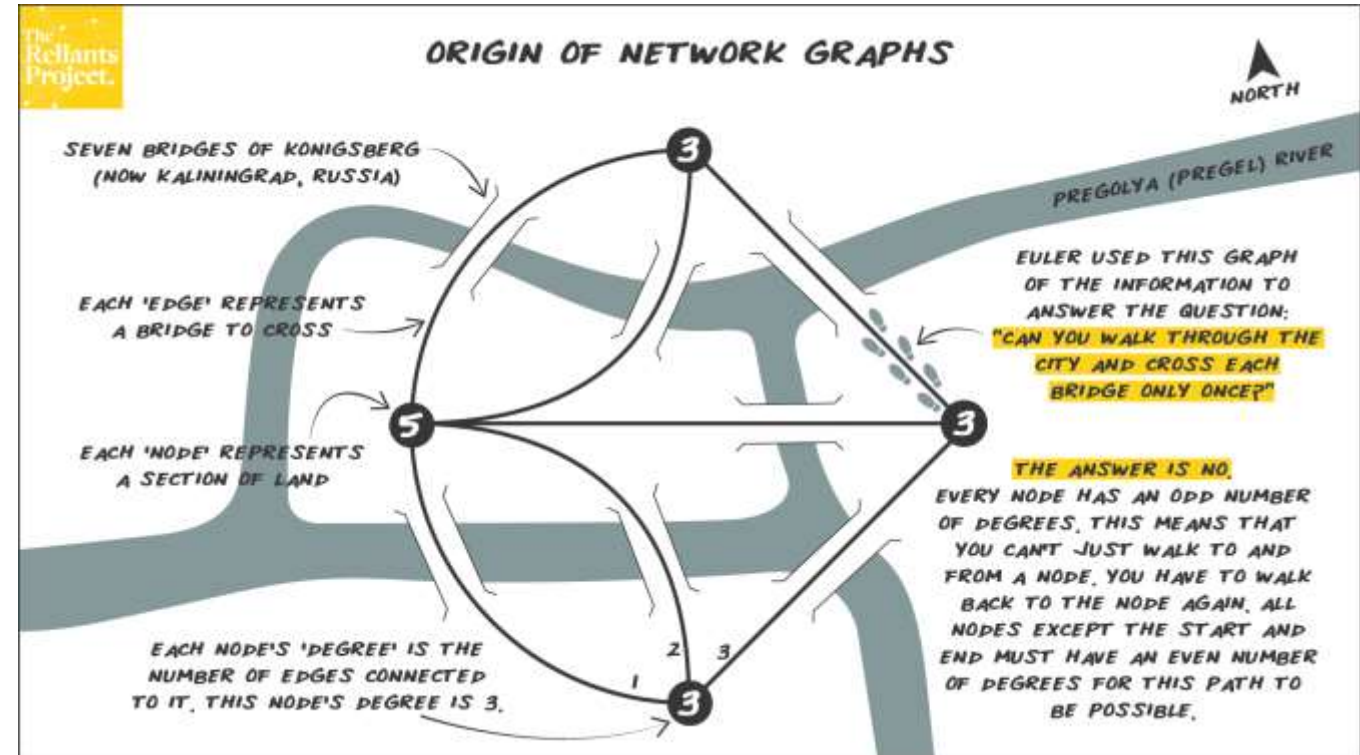
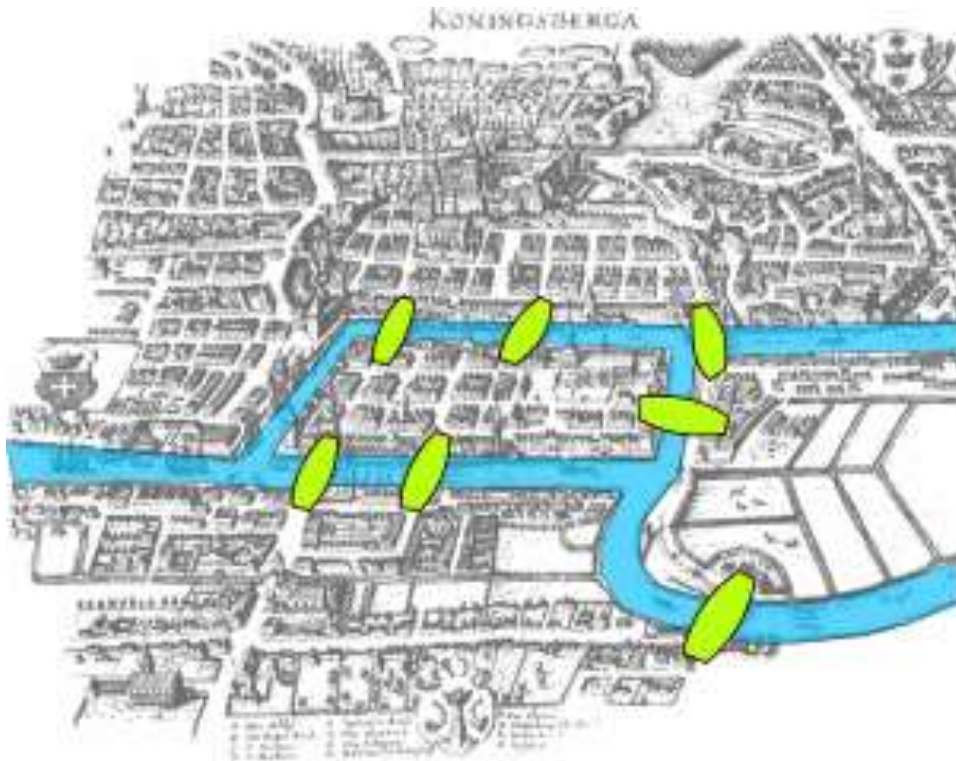
I sette ponti di Königsberg



I sette ponti di Königsberg



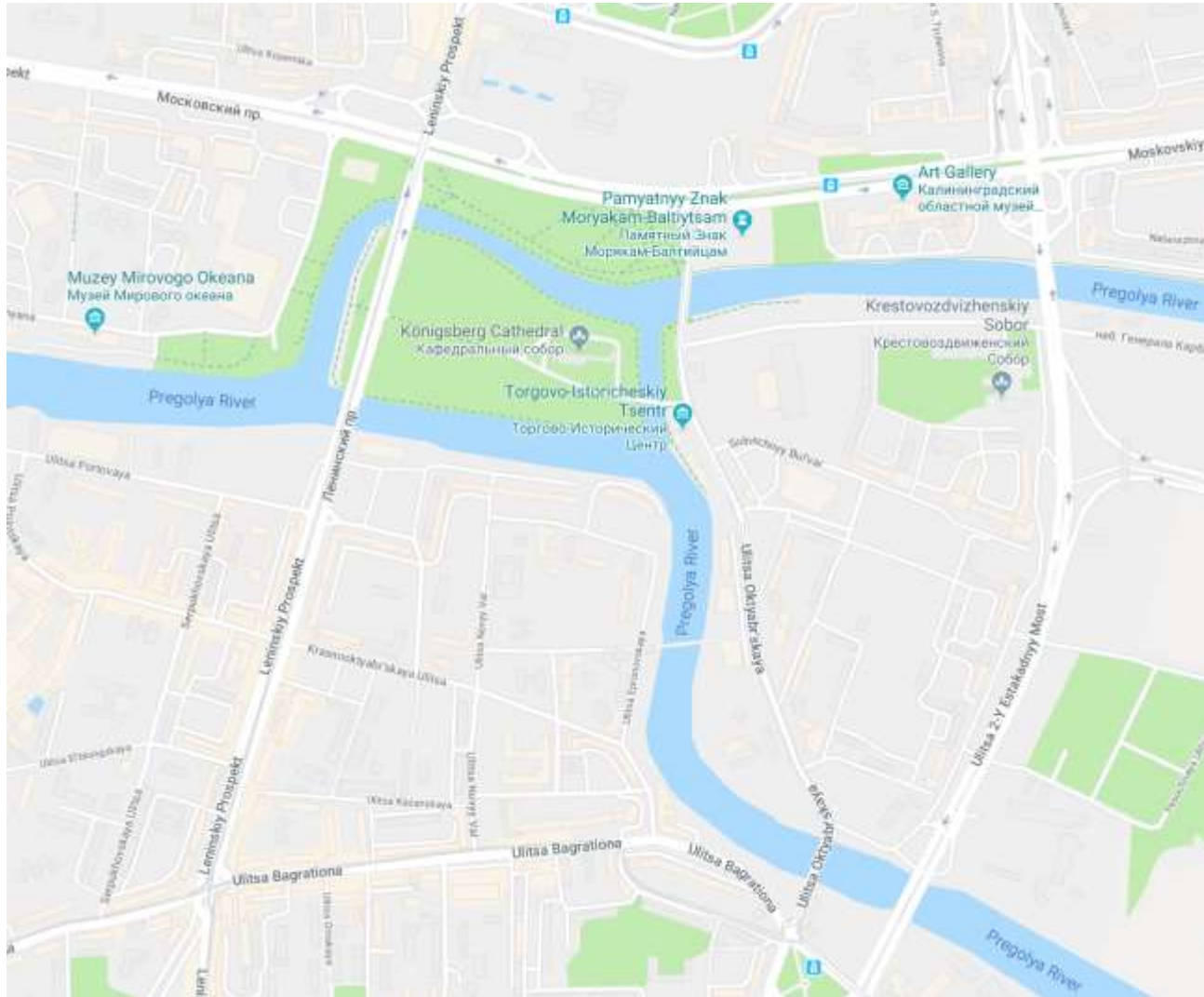
I sette ponti di Königsberg



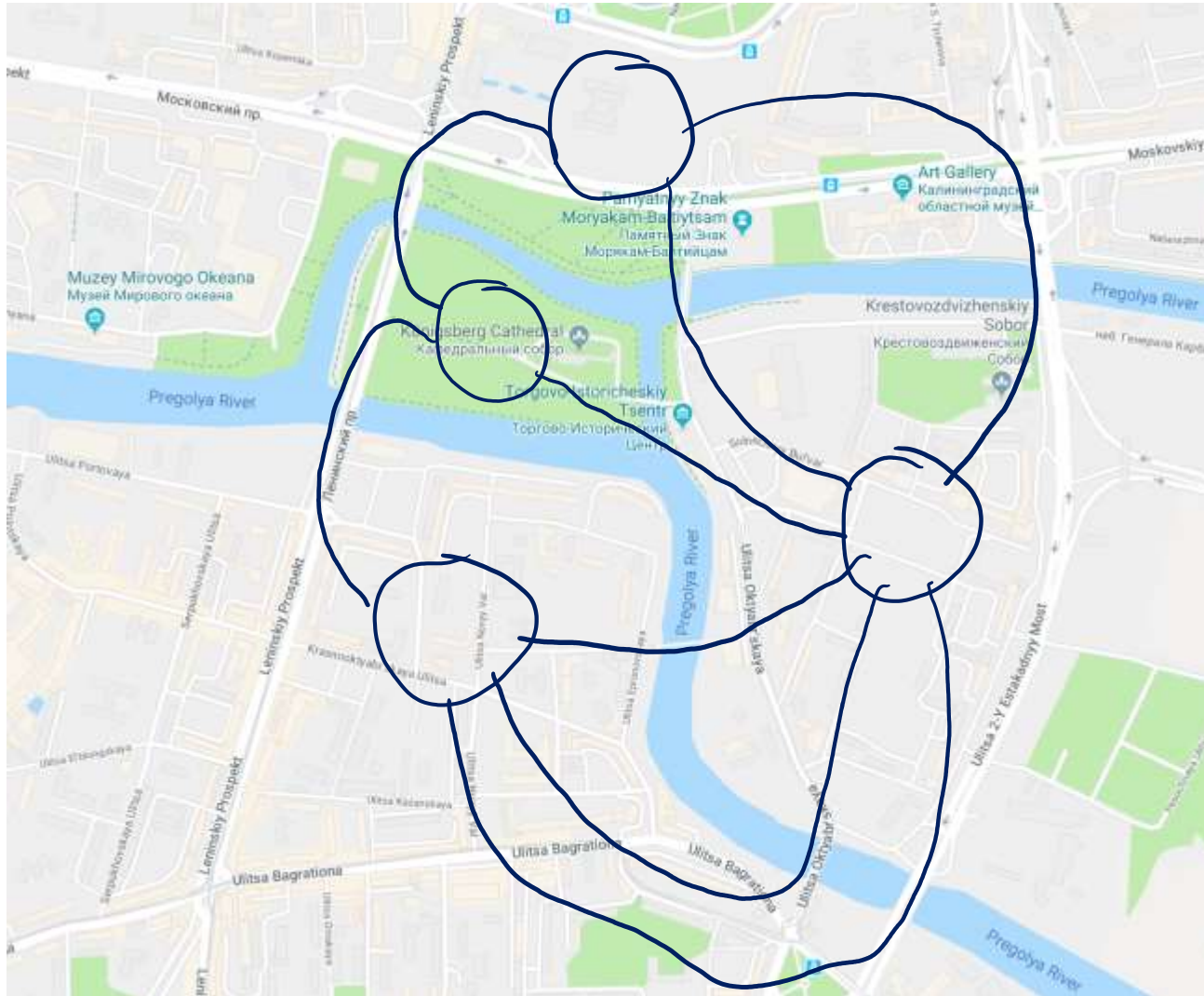
Teorema (Eulero). *Un grafo è Euleriano se e solo se:*

- *Tutti i nodi hanno grado pari, oppure*
- *Soltanto due nodi hanno grado dispari.*

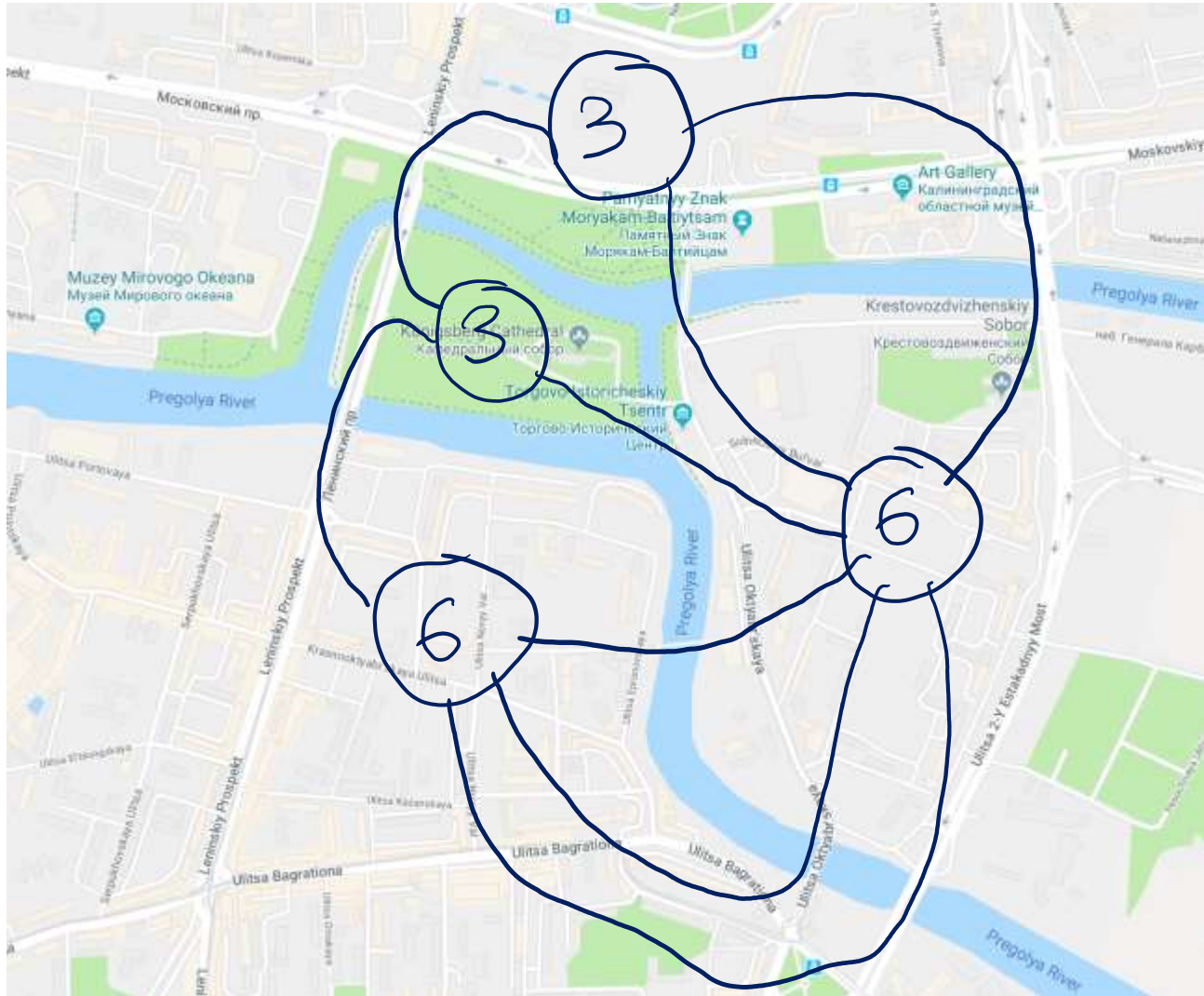
Königsberg oggi: Kaliningrad



Königsberg oggi: Kaliningrad

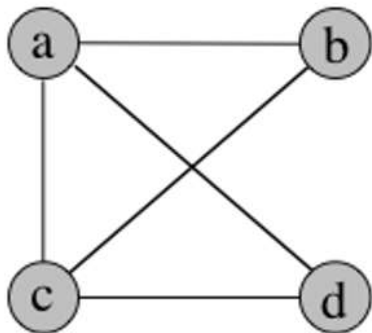


Königsberg oggi: Kaliningrad

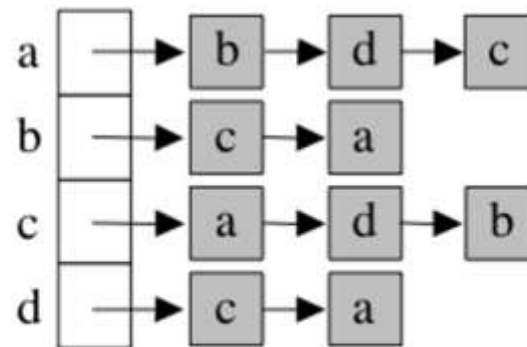


Strutture dati per rappresentare grafi





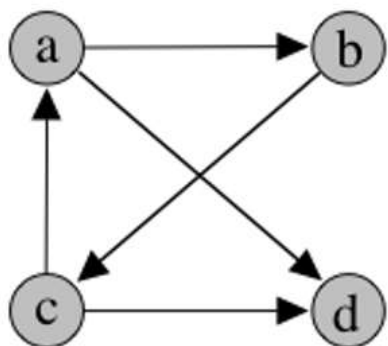
Grafo non orientato G



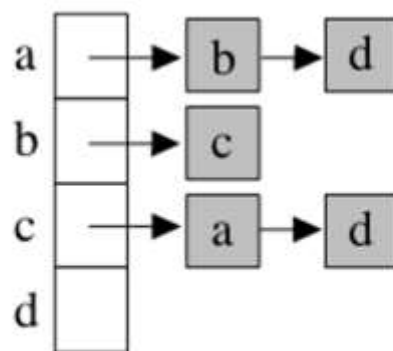
Liste di adiacenza di G

	a	b	c	d
a	0	1	1	1
b	1	0	1	0
c	1	1	0	1
d	1	0	1	0

Matrice di adiacenza di G



Grafo orientato G



Liste di adiacenza di G

	a	b	c	d
a	0	1	0	1
b	0	0	1	0
c	1	0	0	1
d	0	0	0	0

Matrice di adiacenza di G

Principali algoritmi per grafi



Principali algoritmi per grafi

1. Ordinamento topologico (grafi orientati senza cicli)
2. Visita in profondità (DFS) e in ampiezza (BFS)
3. Cammini minimi (Dijkstra)

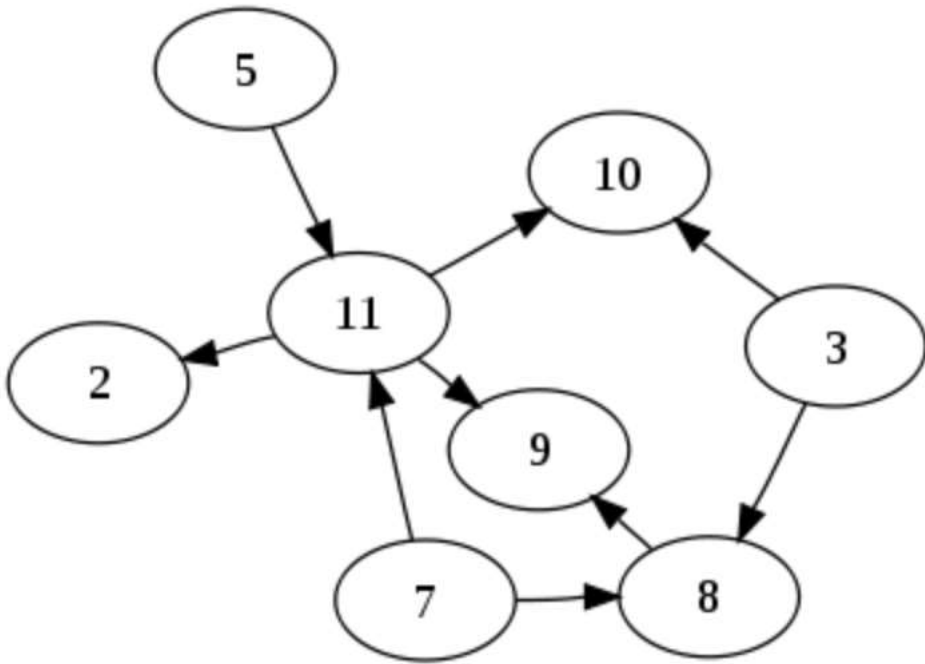
Principali algoritmi per grafi

1. **Ordinamento topologico (grafi orientati senza cicli)**
2. Visita in profondità (DFS) e in ampiezza (BFS)
3. Cammini minimi (Dijkstra)

Grafo diretto aciclico (DAG: Directed Acyclic Graph)

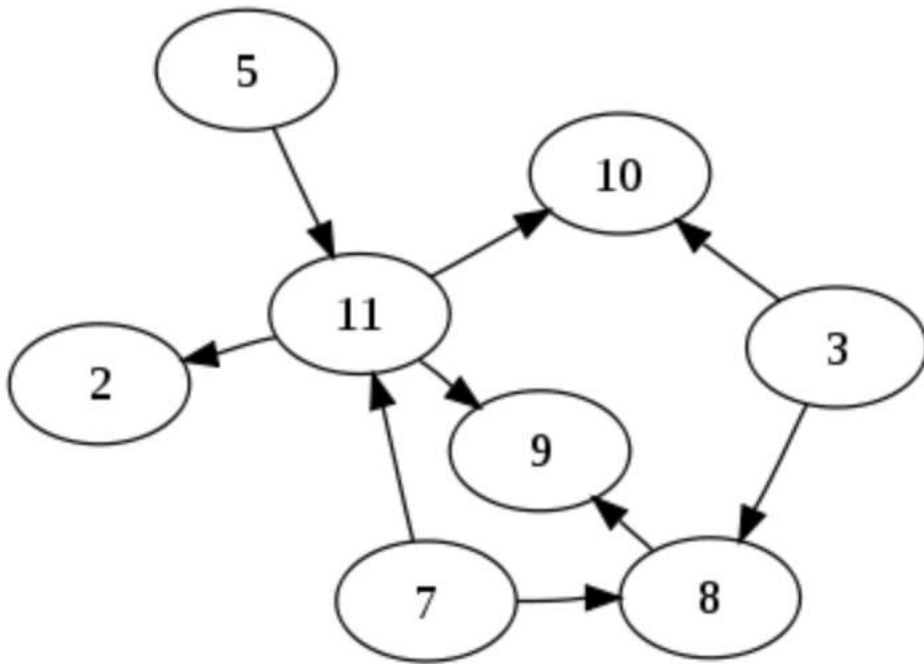
Grafo orientato che non contiene cicli: comunque scegliamo un nodo del grafo, non possiamo mai ritornare allo stesso nodo (percorrendo gli archi del grafo)

Un DAG ammette un ordinamento parziale tra i nodi



Ordinamento topologico

- Se esiste un arco da A a B , allora il nodo A comparirà nell'ordinamento prima di B
- L'ordinamento topologico di un DAG esiste sempre. Non è necessariamente unico



5 3 7 8 11 9 10 2

7 5 11 2 3 10 8 9

7 3 5 11 10 8 2 9

...

Selezioni regionali 2007

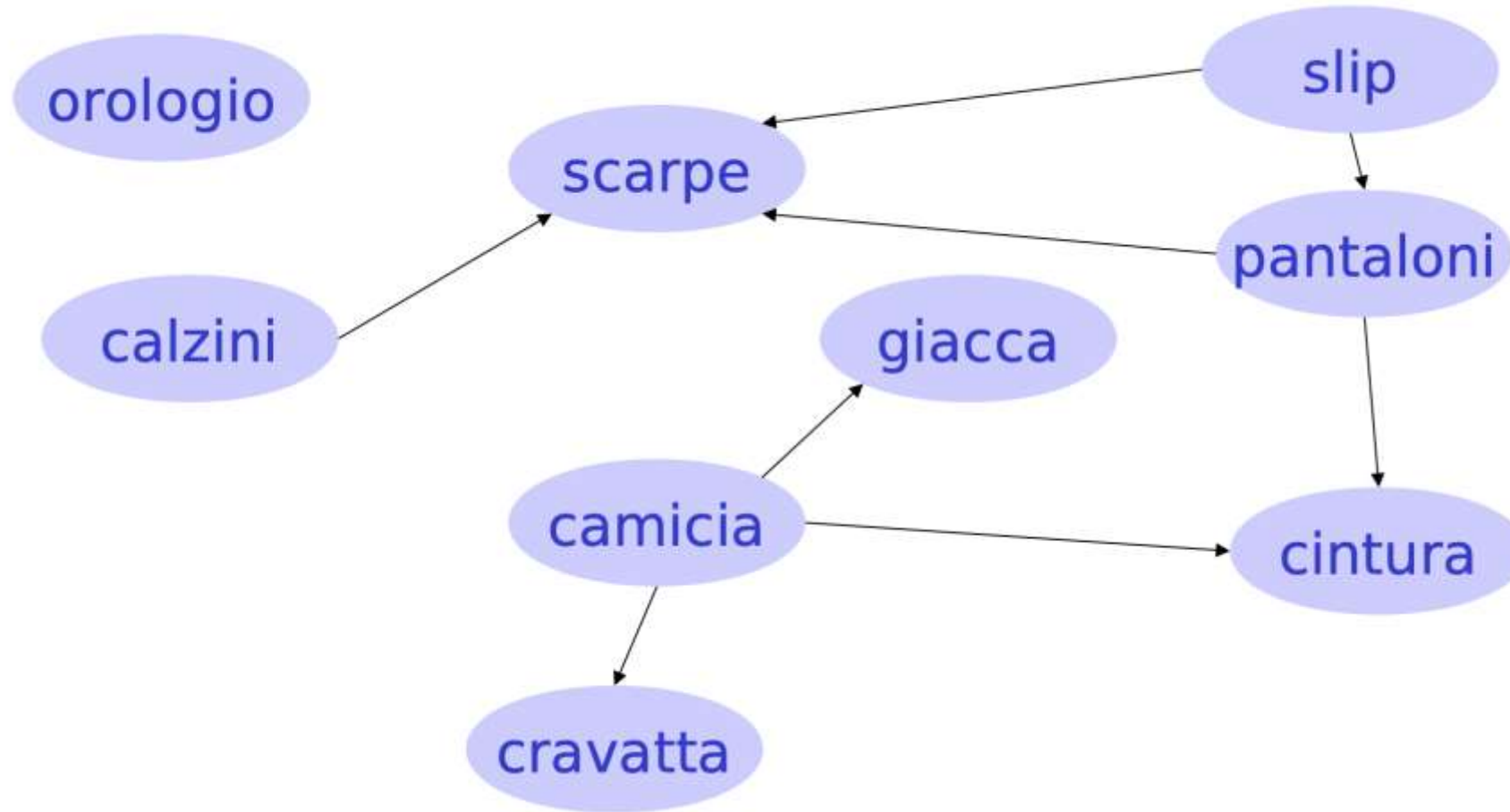
Torero Escamillo (**torero**)

Difficoltà D = 2

Descrizione del problema

Il celebre torero Escamillo deve indossare il proprio costume prima di entrare nell'arena. Egli è costretto a rispettare un dato numero di precedenze, indossando certi indumenti prima di altri, mentre alcuni indumenti possono essere liberamente indossati in un ordine qualsiasi. Per esempio, le "medias" (calze) vanno indossate prima delle "zapatillas" (scarpe), ma non vi è alcun vincolo sull'ordine con cui indossare la "chaquetilla" (giacca) e la "montera" (cappello). Il costume di Escamillo è particolarmente raffinato ed elaborato e si compone di N indumenti. Sfortunatamente, Carmen non ha ancora consegnato uno degli N indumenti necessari alla vestizione di Escamillo. Aiutalo a vestirsi il più possibile, calcolando il massimo numero di indumenti che può indossare in attesa che Carmen gli consegni l'indumento mancante.

Grafo diretto aciclico (DAG: Directed Acyclic Graph)



Dati di input

Il file `input.txt` contiene nella prima riga una tripla di interi, separati da uno spazio: l'intero positivo N che indica il numero di indumenti per la vestizione di Escamillo, dove gli indumenti sono numerati da 1 a N ; l'intero positivo M che indica il numero di precedenze tra coppie di indumenti da rispettare durante la vestizione; l'intero Q , compreso tra 1 e N , che indica l'indumento non ancora consegnato da Carmen. Ognuna delle successive M righe contiene una coppia di interi, compresi tra 1 e N , separati da uno spazio. Tale coppia di interi I e J rappresenta la precedenza in cui l'indumento numero I deve essere indossato prima dell'indumento numero J .

Dati di output

Il file `output.txt` è composto da una riga contenente un solo intero, che rappresenta il massimo numero di indumenti che Escamillo riesce a indossare in attesa dell'indumento Q che Carmen deve ancora consegnargli.

Assunzioni

- $1 < N < 100000$.
- $1 < M < 100000$.
- $1 \leq Q \leq N$.

Esempi di input/output

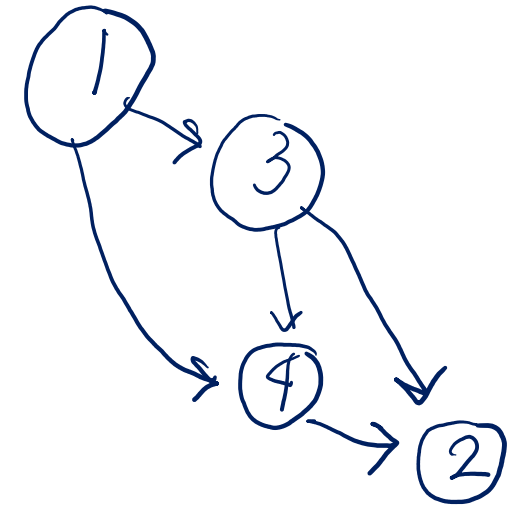
File input.txt	File output.txt
4 5 3 1 3 1 4 3 2 3 4 4 2	1

Assunzioni

- $1 < N < 100000$.
- $1 < M < 100000$.
- $1 \leq Q \leq N$.

Esempi di input/output

File input.txt	File output.txt
4 5 3 1 3 1 4 3 2 3 4 4 2	1

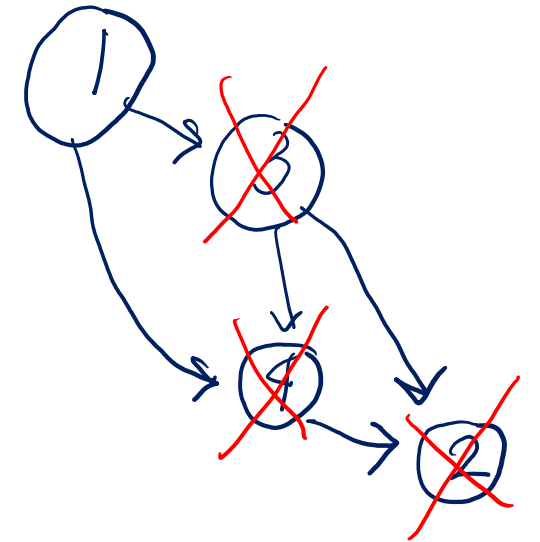


Assunzioni

- $1 < N < 100000$.
- $1 < M < 100000$.
- $1 \leq Q \leq N$.

Esempi di input/output

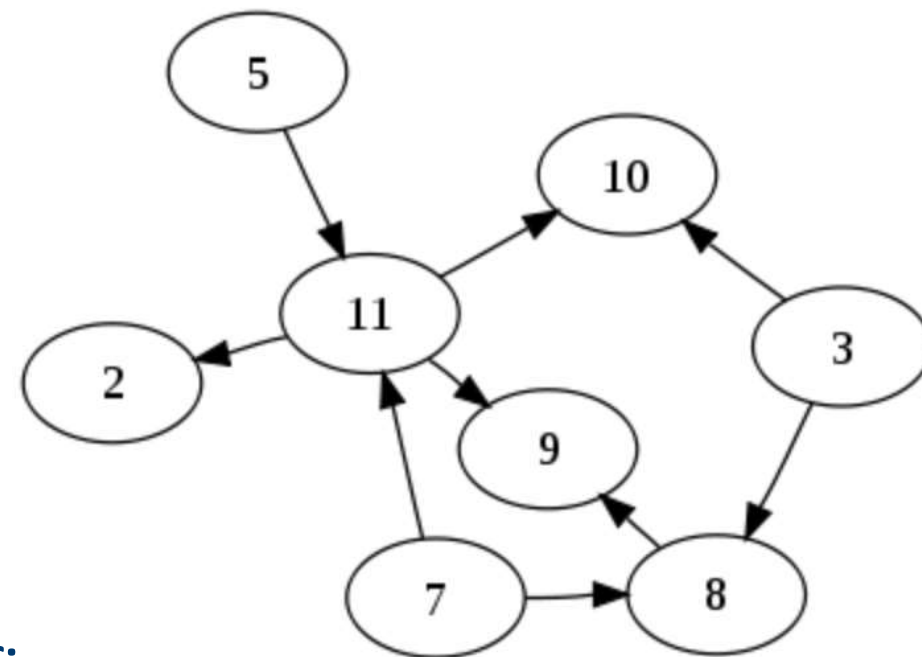
File input.txt	File output.txt
4 5 3 1 3 1 4 3 2 3 4 4 2	1



Algoritmo di ordinamento topologico

1. Trova un nodo che non ha archi entranti, ovvero un nodo sorgente (esiste sempre: perché?)
2. Aggiungilo all'ordinamento topologico
3. Rimuovi dal grafo il nodo e tutti i suoi archi (uscenti)
4. Ripeti... (trova un altro nodo sorgente, ecc)

Se vuoi l'ordinamento, al Passo 2 inserisci alla fine di una lista i nodi sorgente man mano che li incontri



Algoritmo di ordinamento topologico

```
L ← lista vuota che conterrà gli elementi ordinati
S ← insieme di nodi senza archi entranti
while S non è vuoto do
    rimuovi un vertice u da S
    inserisci u in L
    for each vertice v con un arco e da u a v do
        rimuovi arco e dal grafo
        if v non ha altri archi entranti then
            inserisci v in S
if il grafo ha ancora archi then
    ritorna un errore (il grafo ha almeno un ciclo)
else
    ritorna L (l'ordinamento topologico)
```

Complessità lineare:

$O(n+m)$

dove:

- n è il numero di nodi
- m è il numero di archi

Un problema già visto (dalle Territoriali 2015): Rispetta i versi

<https://training.olinfo.it/#/task/disuguaglianze/statement>





Rispetta i versi (disuguaglianze)

Limite di tempo: 1.0 secondi

Limite di memoria: 256 MiB

Difficoltà: 2

Gabriele ha un nuovo rompicapo preferito, chiamato “Rispetta i versi”. Si tratta di un solitario giocato su una griglia formata da N caselle separate da un simbolo di disuguaglianza; in figura è mostrato un esempio con $N = 6$.



L'obiettivo del gioco è quello di riempire le celle vuote con tutti i numeri da 1 a N (ogni numero deve comparire esattamente una volta), in modo da rispettare le disuguaglianze tra caselle adiacenti. Per la griglia della figura, una delle possibili soluzioni al rompicapo è la seguente:



Dati di input

Il file `input.txt` contiene due righe di testo. Sulla prima è presente l'intero N , il numero di caselle del gioco. Sulla seconda è presente una stringa di $N - 1$ caratteri, ognuno dei quali può essere solo `<` o `>`, che descrive i vincoli tra le caselle, da sinistra a destra.

Dati di output

Il file `output.txt` contiene su una sola riga una qualunque permutazione dei numeri da 1 a N - separati tra loro da uno spazio - che risolve il rompicapo. I numeri corrispondono ai valori scritti nelle caselle, leggendo da sinistra verso destra.

Assunzioni

- $2 \leq N \leq 100\,000$.
- Nel 30% dei casi, il valore di N non supera 10.
- Nel 60% dei casi, il valore di N non supera 20.
- Si garantisce l'esistenza di almeno una soluzione per ciascuno dei casi di test utilizzati nella verifica del funzionamento del programma.

Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

Esempi di input/output

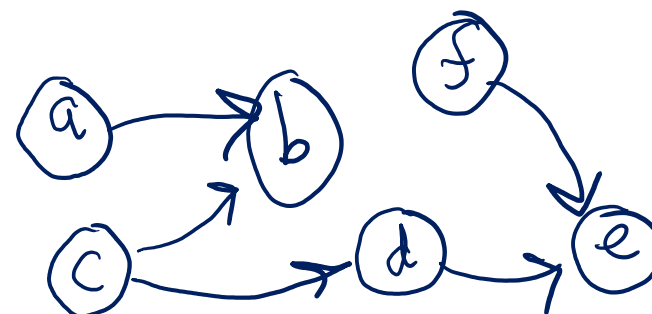
input.txt	output.txt
6 <><>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

$a < b > c < d < e > f$

Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

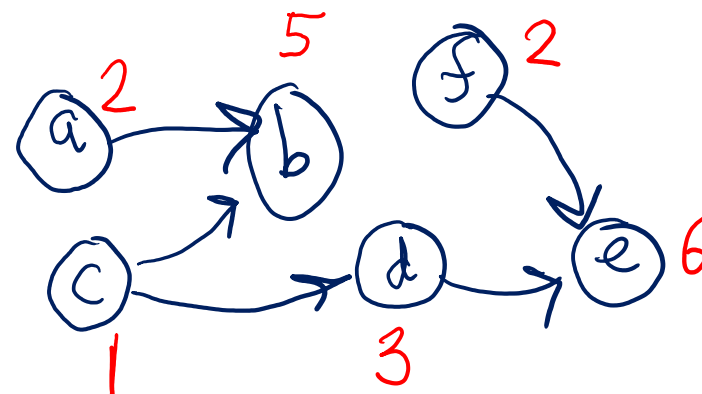
$a < b > c < d < e > f$



Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

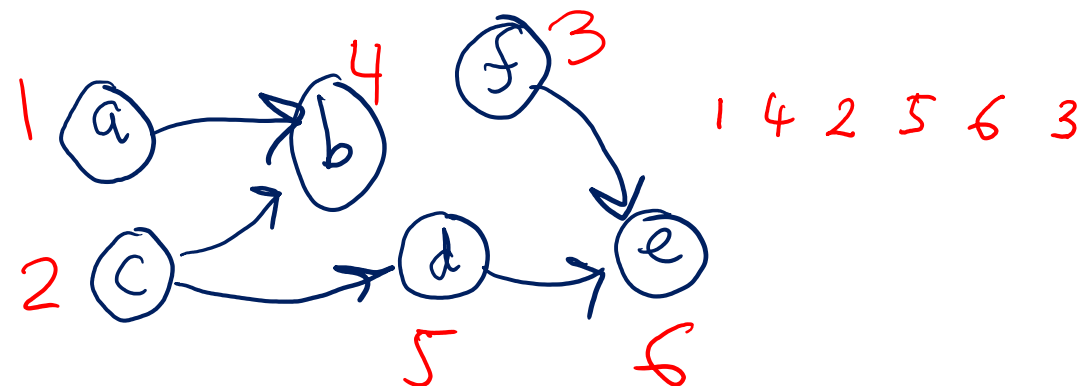
$a < b > c < d < e > f$



Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

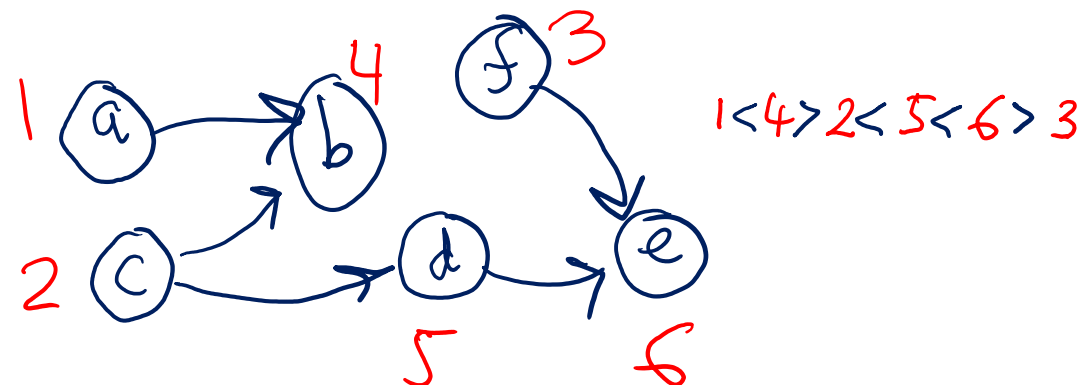
$a < b > c < d < e > f$
(altra soluzione)



Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

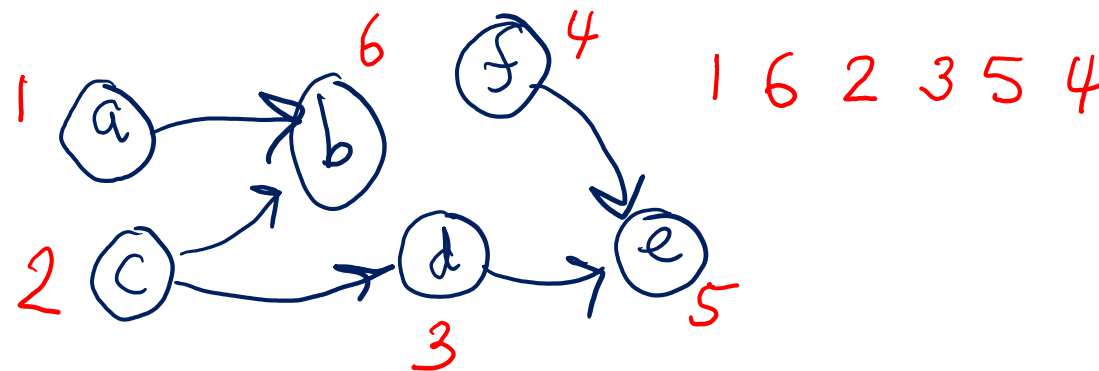
$a < b > c < d < e > f$
(altra soluzione)



Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

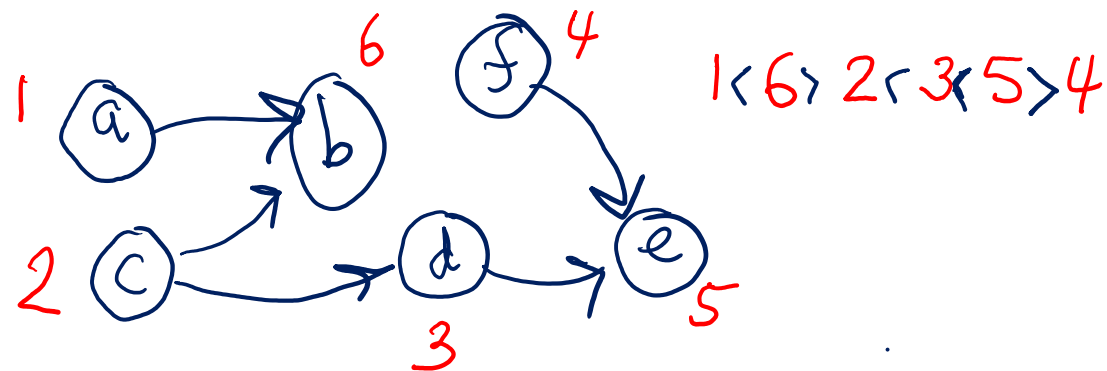
$a < b > c < d < e > f$
(Soluzione dell'altra volta!)



Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

$a < b > c < d < e > f$
(Soluzione dell'altra volta!)



Esempi di input/output

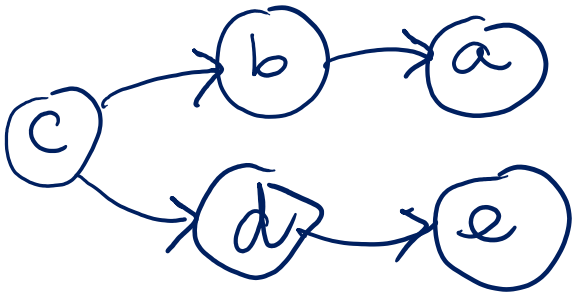
input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

a>b>c<d<e

Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

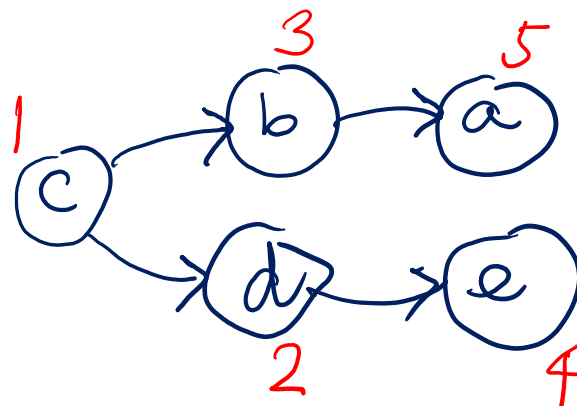
a>b>c<d<e



Esempi di input/output

input.txt	output.txt
6 <><<>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

$a > b > c < d < e$

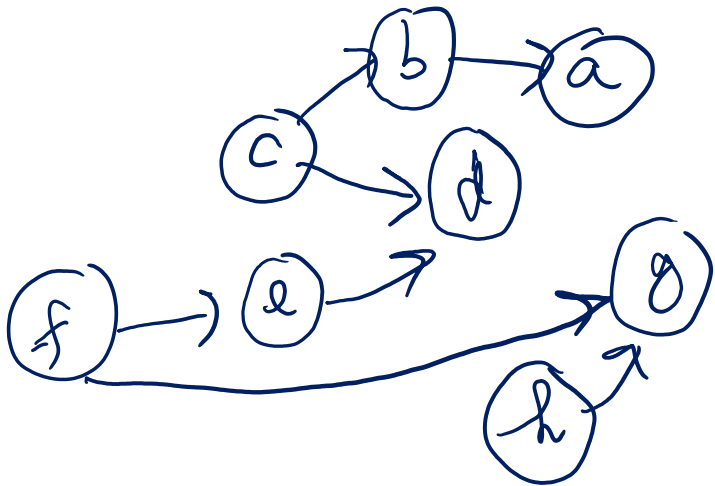


Esempi di input/output

input.txt	output.txt
6 <><>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

→

a>b>c<d>e>f<g>h

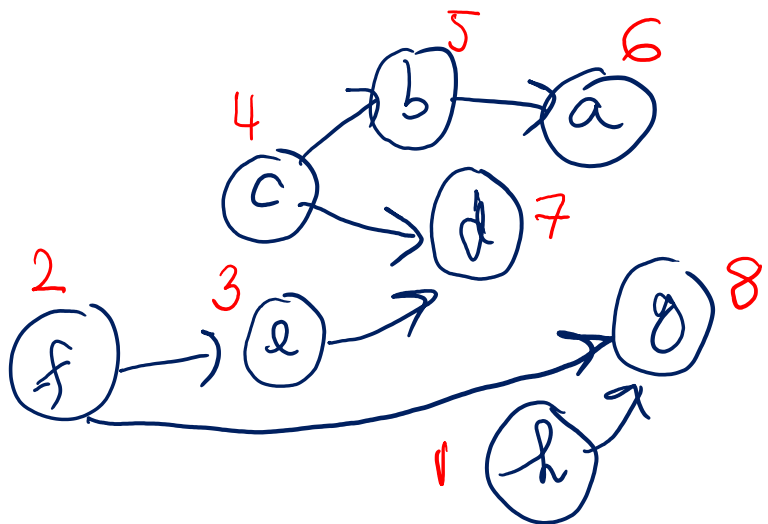


Esempi di input/output

input.txt	output.txt
6 <><>	2 5 1 3 6 4
5 >><<	5 3 1 2 4
8 >><>><>	6 5 4 7 3 2 8 1

→

a>b>c<d>e>f<g>h



Sunnydale (Territoriali 2004)

<https://training.olinfo.it/#/task/sunny/statement>



Sunnydale (sunny)

Difficoltà $D = 2$.

Descrizione del problema

Sunnydale è una città che - per ragioni storiche e ambientali - ospita un elevatissimo numero di vampiri. Per ragioni cutanee i vampiri non possono sopportare la luce solare e, storicamente, hanno sempre avuto enormi difficoltà a viaggiare col sole alto nel cielo; l'attraversamento delle gallerie sotterranee di Sunnydale è sempre stato il mezzo preferito dai vampiri per muoversi nella città. I continui crolli delle gallerie hanno creato dei fori nei soffitti, rendendone alcune troppo luminose per un attraversamento tranquillo e sereno. Harmony, una ragazza-vampiro, passeggia per le gallerie di Sunnydale quando il suo amico Spike le telefona per invitarla a casa sua. Purtroppo ella si muove per le gallerie sotterranee secondo una regola tanto semplice quanto tassativa: ad ogni svincolo sceglie sempre e comunque la galleria meno luminosa per paura di rovinare la propria pelle. Sapendo che non esistono due gallerie egualmente luminose, bisogna determinare se Harmony possa raggiungere la casa sotterranea di Spike e, in caso affermativo, quante gallerie le sono necessarie per arrivare.



Dati di input

La prima riga del file `input.txt` è composta da quattro numeri interi N , M , H e S : il primo rappresenta il numero degli svincoli (numerati da 1 a N), il secondo rappresenta il numero delle gallerie, il terzo rappresenta l'indice dello svincolo in cui si trova Harmony quando riceve la telefonata; il quarto, infine, rappresenta l'indice dello svincolo della casa di Spike. Ognuna delle successive M righe descrive una galleria e contiene tre numeri interi A , B e L separati da uno spazio: i primi due rappresentano gli svincoli collegati dalla galleria mentre il terzo rappresenta il suo grado di luminosità.

Dati di output

Il file `output.txt` dovrà contenere un unico numero intero: -1 se Harmony non riuscirà a raggiungere Spike; altrimenti, il numero di gallerie che ella percorrerà prima di raggiungerlo.

Assunzioni

- $2 \leq N \leq 50000$
- $1 \leq M \leq 50000$
- Non esistono due gallerie con la stessa luminosità L .
- Per ogni galleria, $1 \leq L \leq M$.
- $1 \leq H, S \leq N$

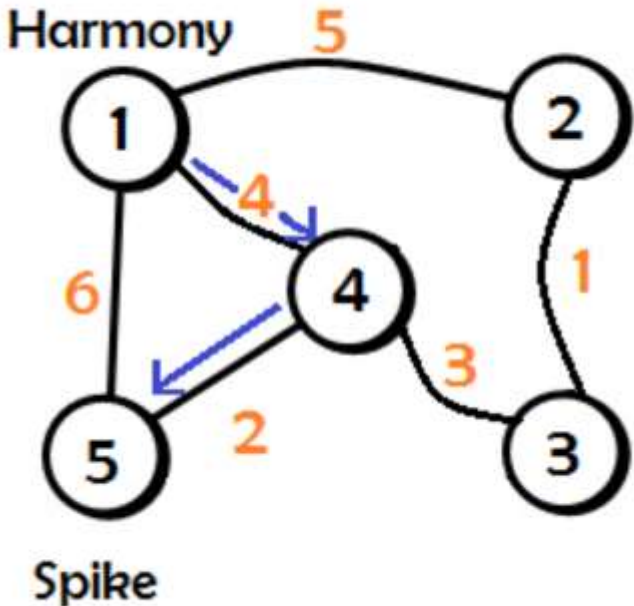
Esempi di input/output

File input.txt

5 6 1 5
1 2 5
2 3 1
3 4 3
4 5 2
5 1 6
1 4 4

File input.txt

3 2 2 1
3 1 2
2 3 1



File output.txt

2

File output.txt

-1

Riflessioni

- Ci serve proprio memorizzare tutti gli archi?
- Come memorizziamo il grafo?
(Ogni nodo ha un solo arco uscente)

Riflessioni

- Come facciamo a non ritornare sullo stesso nodo per evitare di ciclare all'infinito?

Monete a posto (Gator 2016)

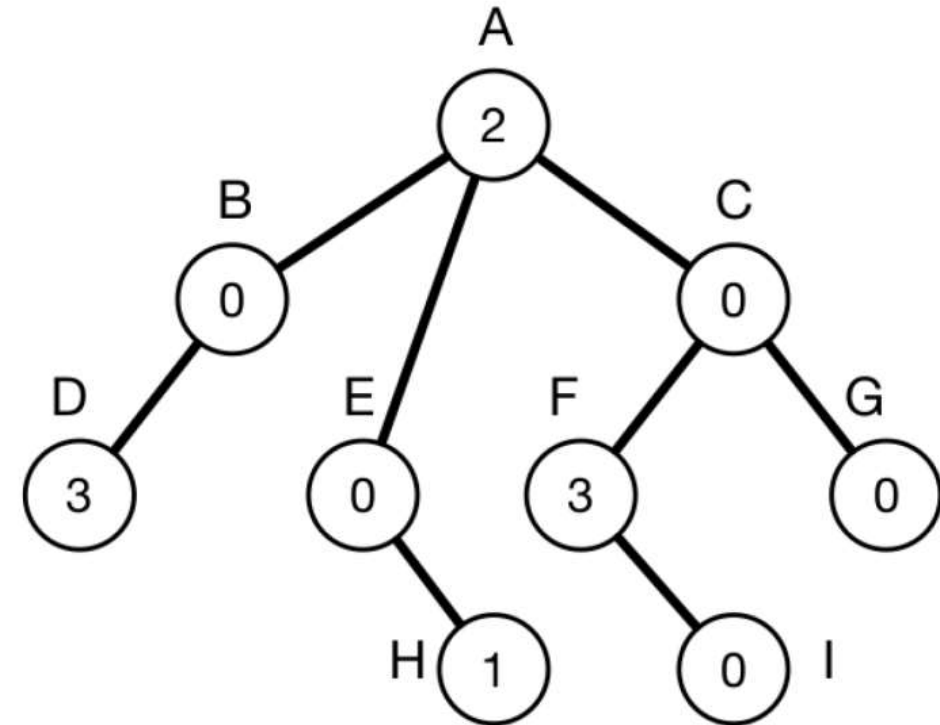
https://training.olinfo.it/#/task/gator_monete/statement





GATOR – Gara Allenamento TOR Vergata 2016

Gara online, 9-10 aprile 2016



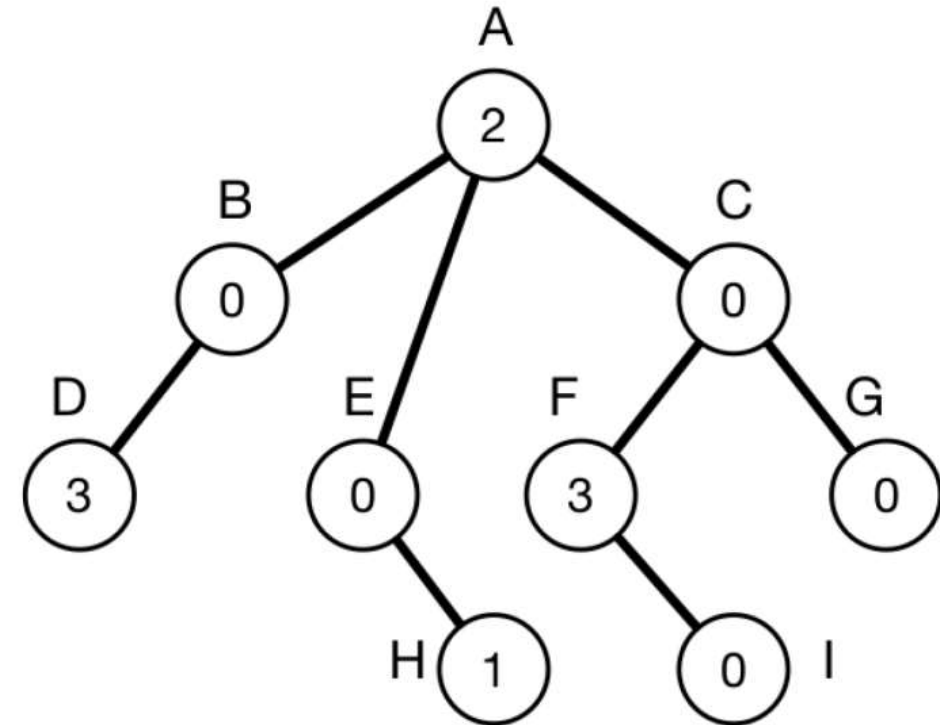
Monete a posto (monete)

William colleziona monete, e le tiene in un espositore come quello mostrato in figura, in cui ci sono N contenitori numerati da 1 a N appesi uno all'altro tramite una barra di metallo. In particolare, ogni contenitore tranne quello più in alto (il contenitore numero 1) è appeso a esattamente un altro contenitore. Sua sorella ha giocato con le monete, che prima erano una in ogni posizione, e le ha lasciate nella situazione mostrata qui sotto. William ha però una regola per rimetterle a posto: può spostare una sola moneta per volta, da un contenitore a uno collegato. Quante operazioni sono sufficienti a William per fare in modo che in ogni contenitore ci sia una moneta?

Ad esempio, nella situazione mostrata qui sopra, William impiega:

- Una operazione per spostare una moneta da A a E.
- Una operazione per spostare una moneta da F a I.
- Una operazione per spostare una moneta da F a C.
- Una operazione per spostare una moneta da D a B.
- Quattro operazioni per spostare una moneta da D a G (passando per B, A e infine C).

Dall'esempio qui sopra si vede che servono otto operazioni in totale per rimettere a posto le monete, con una moneta per ogni contenitore.



Dati di input

Il file `input.txt` contiene tre righe. La prima riga contiene l'intero N , il numero di contenitori (e di monete) che ha William. La seconda riga contiene i numeri interi $a_1, \dots, a_N$, che rappresentano il numero di monete contenute nel rispettivo contenitore dopo che queste sono state disordinate. Infine la terza riga contiene $N - 1$ interi $b_2, \dots, b_N$ che indicano che il contenitore j è appeso al contenitore b_j .

Dati di output

Sul file `output.txt` stampare il numero minimo di spostamenti necessari a William per rimettere a posto tutte le monete.

Assunzioni

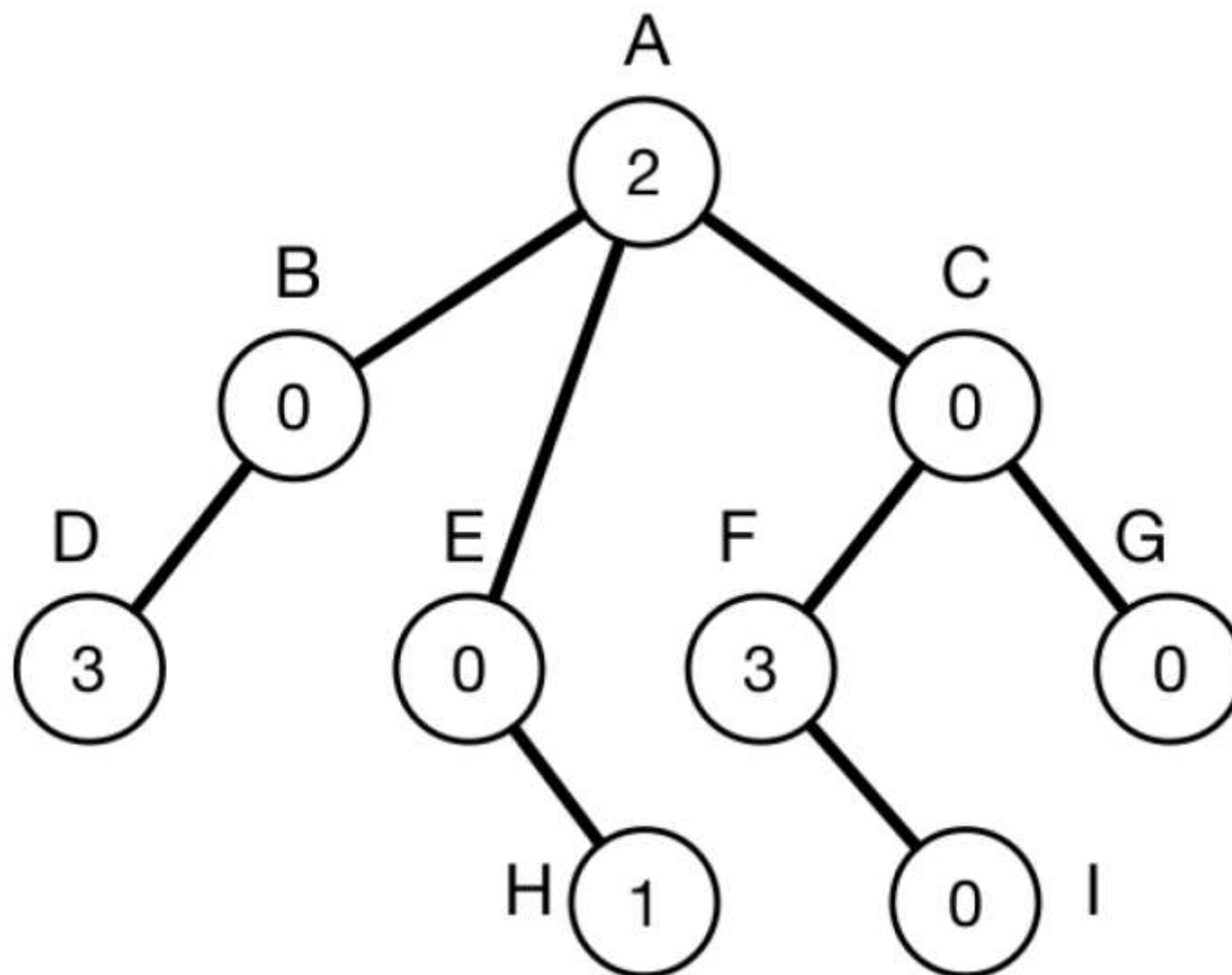
- $1 \leq N \leq 100\,000$.
- $0 \leq a_i \leq N$ per ogni $i = 1, \dots, N$.
- $a_1 + \dots + a_N = N$.
- $1 \leq b_i < i$ per ogni $i = 2, \dots, N$.

Esempi di input/output

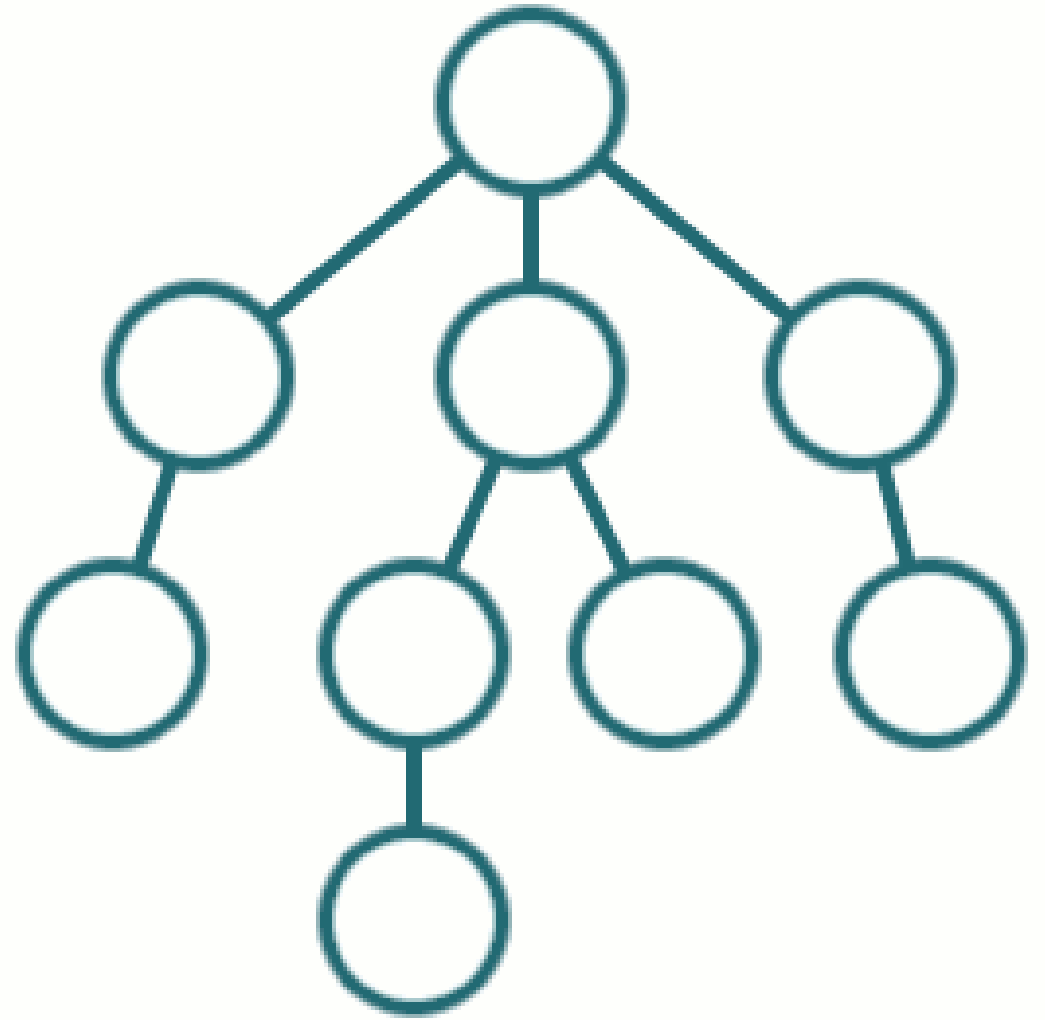
input.txt	output.txt
9 2 0 0 3 0 3 0 1 0 1 1 2 1 3 3 5 6	8
10 0 0 1 2 1 1 2 0 3 0 1 2 2 2 4 3 5 1 8	11

Esempi di input/output

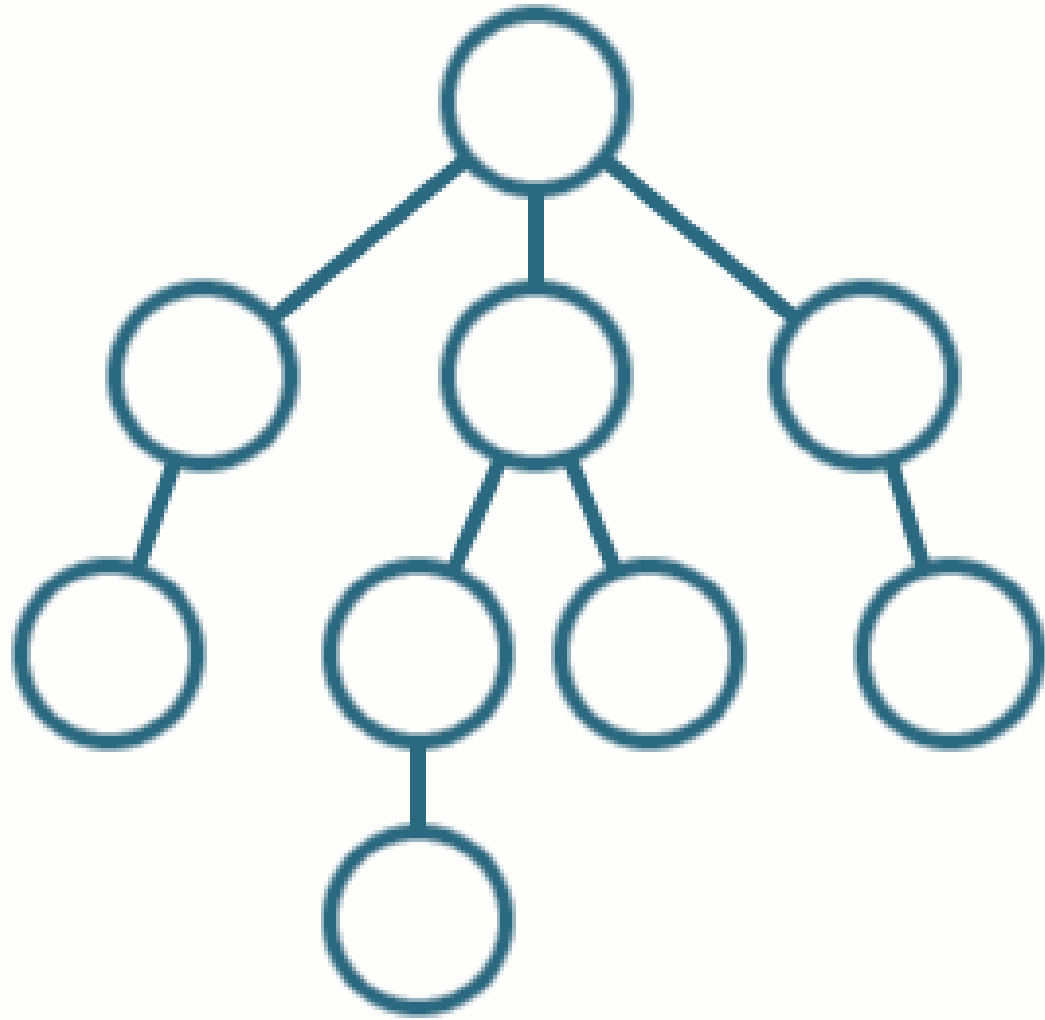
input.txt	output.txt
9 2 0 0 3 0 3 0 1 0 1 1 2 1 3 3 5 6	8
10 0 0 1 2 1 1 2 0 3 0 1 2 2 2 4 3 5 1 8	11



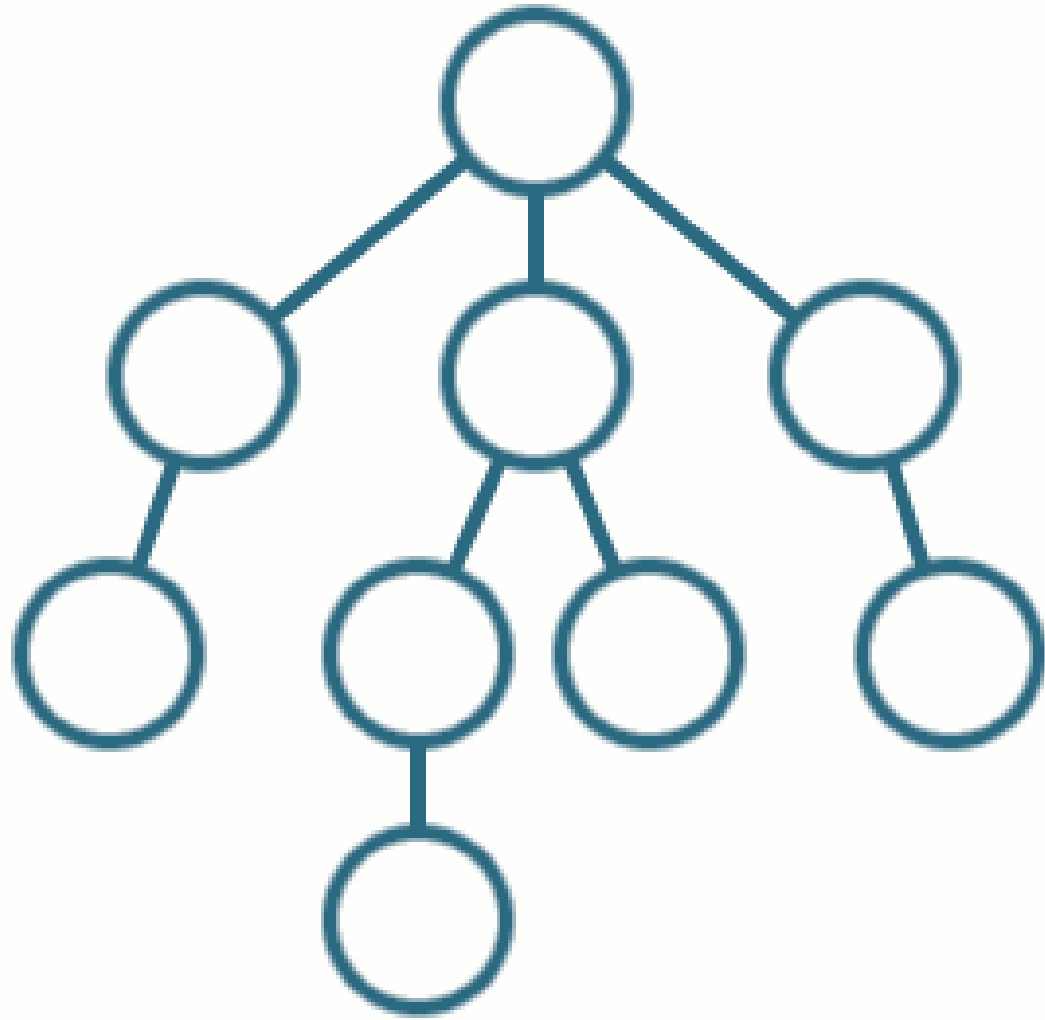
BFS



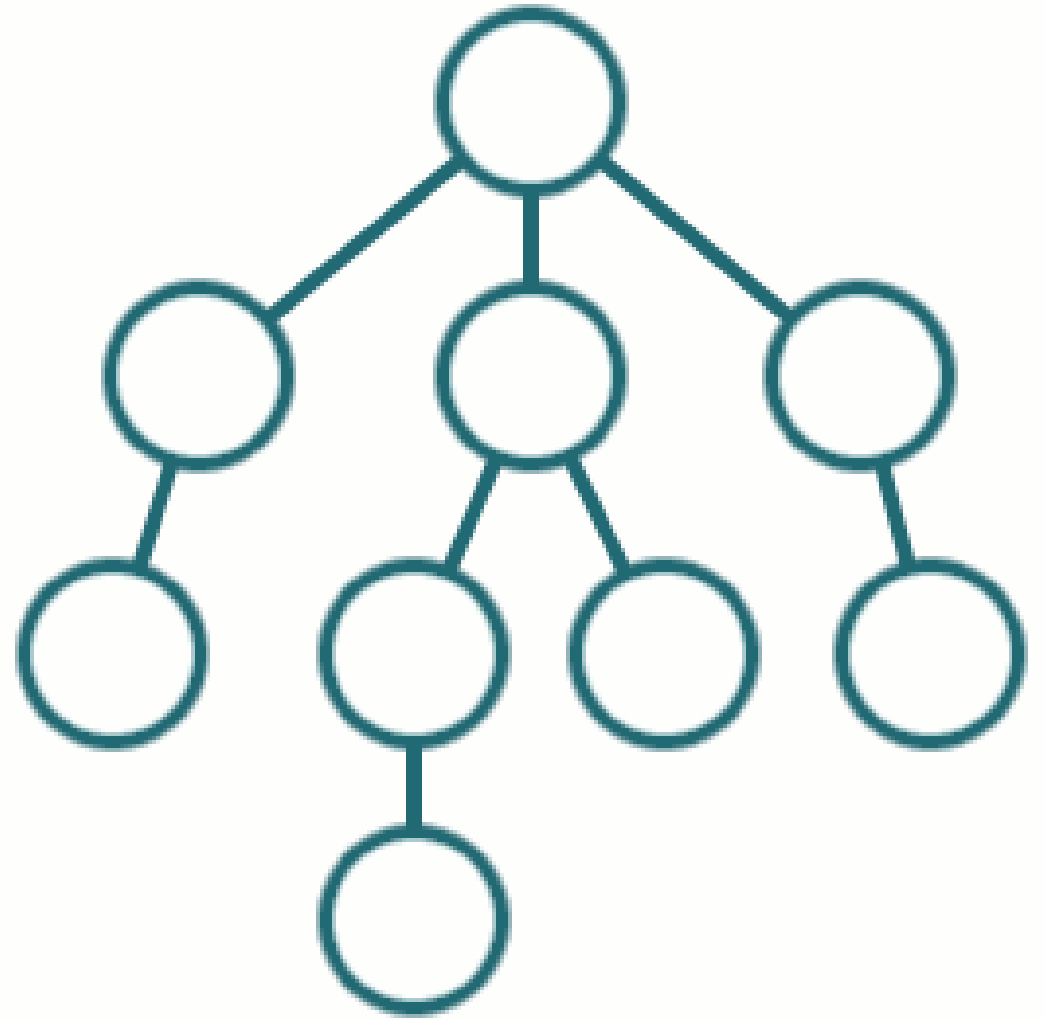
DFS



DFS



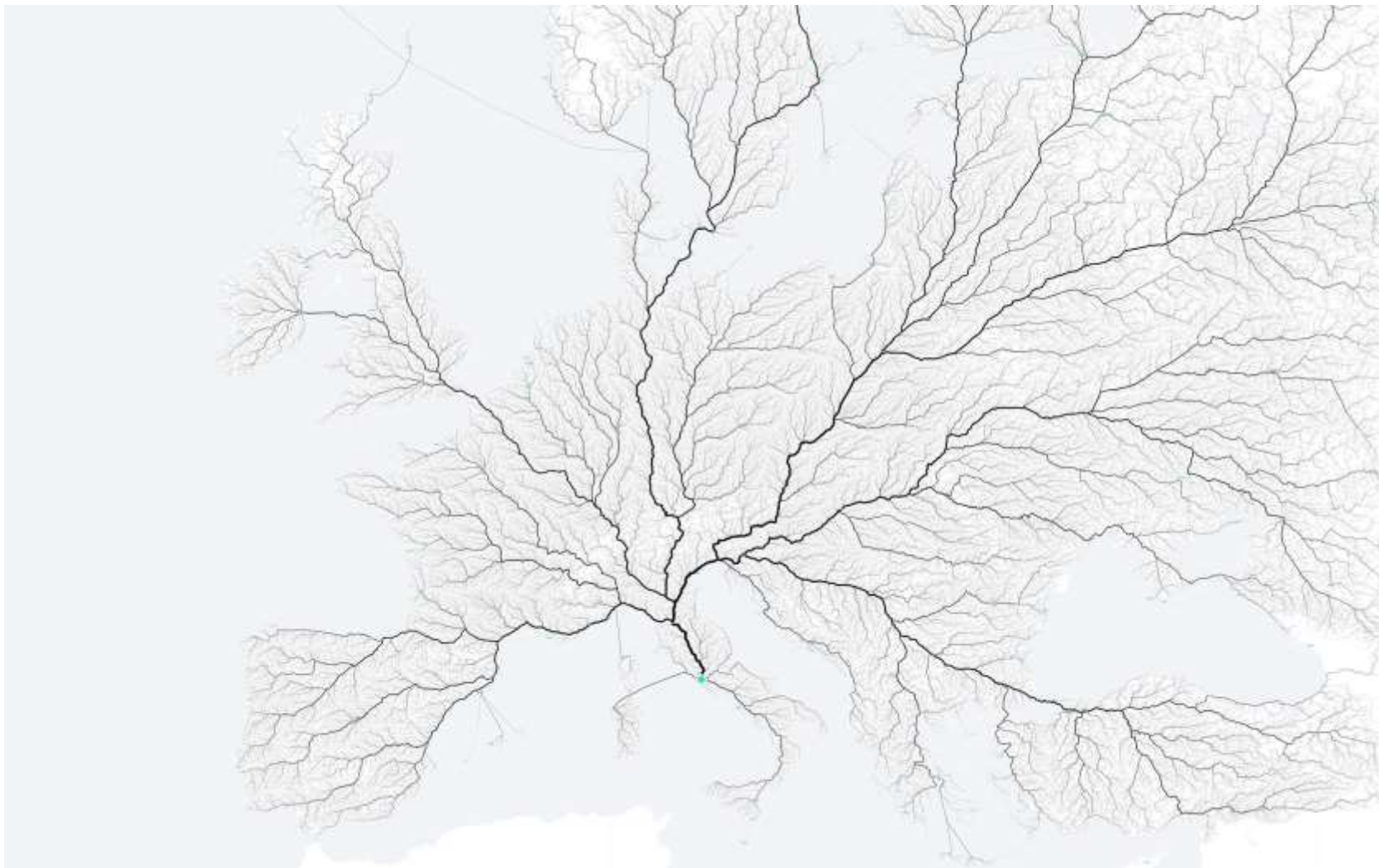
BFS



BFS /DFS (prossima lezione)



Cammini minimi (prossima lezione)



Cammini minimi (Dijkstra)

Bae: Come over

Dijkstra: But there are so many routes to take and
I don't know which one's the fastest

Bae: My parents aren't home

Dijkstra:

Dijkstra's algorithm

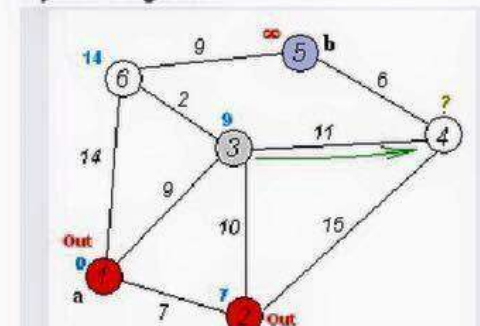
Graph search algorithm

Not to be confused with Dykstra's projection algorithm.

Dijkstra's algorithm is an [algorithm](#) for finding the [shortest paths](#) between [nodes](#) in a [graph](#), which may represent, for example, road networks. It was conceived by [computer scientist Edsger W. Dijkstra](#) in 1956 and published three years later.^{[1][2]}

The algorithm exists in many variants; Dijkstra's original variant found the shortest path between two nodes,^[2] but a more common variant fixes a single node as the "source" node and finds shortest paths from the source to all other nodes in the graph, producing a [shortest-path tree](#).

Dijkstra's algorithm



Compiti a casa (Esercizi Olimpiadi)



Compiti a casa

Prova a risolvere i seguenti problemi:

- Splitting the bill https://training.olinfo.it/#/task/ois_money/statement
- Water park https://training.olinfo.it/#/task/ois_waterslide/statement
- Appetito aracnide <https://training.olinfo.it/#/task/tecla/statement>
- Piano di studi https://training.olinfo.it/#/task/luiss_piano/statement
- Incendia lo trabucco https://training.olinfo.it/#/task/po_trabucco/statement

